



# ESTRUCTURAS DE DATOS PROYECTO FINAL

**Alumno:**

Calixto Isaac Galeana Medrano

**Matricula:**

7205504

**Carrera:**

Ingenieria en Desarrollo de Software

**Tutoria:**

Joel Balderrama

## **Tabla de contenido**

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Descripción y Objetivos de la actividad .....</b>      | <b>3</b>  |
| <b>Entregables Contemplados para esta actividad .....</b> | <b>3</b>  |
| <b>Instrucciones .....</b>                                | <b>4</b>  |
| <b>Documento Técnico .....</b>                            | <b>5</b>  |
| <b>Datos del proyecto .....</b>                           | <b>5</b>  |
| <b>Descripción General del Sistema .....</b>              | <b>6</b>  |
| <b>Objetivos .....</b>                                    | <b>8</b>  |
| <b>Documento de Prueba de Ejecución .....</b>             | <b>43</b> |
| <b>Conclusión .....</b>                                   | <b>75</b> |
| <b>Referencias Bibliográficas.....</b>                    | <b>77</b> |

## **Descripción y Objetivos de la actividad**

### **Descripción**

Los aprendedores desarrollarán un sistema avanzado de gestión de tareas para una empresa, el cual optimizará la asignación, búsqueda, y organización de las tareas mediante el uso de estructuras de datos complejas como colas de prioridad, árboles binarios, y grafos. Además, implementarán funciones de búsqueda y ordenamiento para gestionar los datos de manera eficiente.

### **Objetivo**

Diseñar e implementar un sistema de gestión de tareas mediante estructuras de datos avanzadas y algoritmos de ordenamiento y búsqueda para optimizar las operaciones de asignación, consulta, y organización de tareas.

## **Entregables Contemplados para esta actividad**

Entrega un documento en Word con los resultados de tu actividad.

- Código fuente del sistema de gestión de tareas.
- Explicación sobre cómo se utilizaron las estructuras de datos y algoritmos.
- Capturas de pantallas que demuestren el uso del sistema con ejemplos de asignación de tareas, búsqueda, y organización.

# Instrucciones

## 1. Preparación:

- a. Revisa los conceptos de las estructuras de datos y algoritmos mencionados.
- b. Revisar la documentación de la fase I del proyecto.

## 2. Implementación de la cola de prioridades:

- a. Crea una cola de prioridades para gestionar las tareas con base en su urgencia y fecha de entrega.

## 3. Uso de árboles binarios:

- a. Implementa árboles binarios para organizar los empleados por departamentos y optimizar la búsqueda de asignaciones .

## 4. Recursividad y algoritmos “divide y vencerás”:

- a. Aplica la recursividad para calcular estadísticas de las tareas, como el tiempo estimado para completarlas.
- b. Utiliza el concepto de “divide y vencerás” para desarrollar un algoritmo que optimice la distribución de tareas en proyectos grandes .

## 5. Tablas *hash* y clase *hashmap*:

- a. Emplea tablas *hash* para almacenar y acceder rápidamente a la información de tareas y empleados.

## 6. Métodos de ordenamiento y búsqueda:

- a. Implementa métodos de ordenamiento para organizar las tareas por prioridad y fecha de entrega.
- b. Desarrolla métodos de búsqueda eficientes para encontrar tareas específicas.

## 7. Grafos:

- a. Utiliza grafos para representar las dependencias entre tareas en proyectos complejos.

## Documento Técnico

Sistema Empresarial Avanzado de Gestión de Tareas (GestionTareasApp)

### Datos del proyecto

| Campo                   | Detalle                                                                                               |
|-------------------------|-------------------------------------------------------------------------------------------------------|
| Nombre del proyecto     | Sistema Empresarial Avanzado de Gestión de Tareas                                                     |
| Nombre del paquete raíz | `com.example`                                                                                         |
| Materia                 | Estructuras de Datos                                                                                  |
| Tipo de entrega         | Proyecto Final                                                                                        |
| Lenguaje / Plataforma   | Java 21 (maven.compiler.release = 21), interfaz gráfica Swing                                         |
| Gestor de dependencias  | Apache Maven (artefacto `demo`, `groupId: com.example`)                                               |
| Base de datos           | SQLite, mediante el controlador JDBC org.xerial:sqlite-jdbc:3.46.1.0 (archivo data/gestion_tareas.db) |
| Patrón de arquitectura  | Modelo - Vista - Controlador (MVC) con una capa adicional de Persistencia (repositorios DAO)          |

## Descripción General del Sistema

El sistema es una aplicación de escritorio (Java Swing) que simula la gestión de tareas dentro de una organización, integrando en una sola interfaz gráfica múltiples estructuras de datos y algoritmos vistos a lo largo del curso. Permite:

- Registrar tareas asignándolas a distintas estructuras según su naturaleza: **Pila** (urgentes), **Cola** (programadas), **Lista** (generales) o **Cola de Prioridad** (ordenadas por urgencia y fecha de entrega).
- Asignar opcionalmente a cada tarea un **responsable directo**, que solo puede ser un empleado del mismo departamento de la tarea.
- Administrar empleados mediante un **Árbol Binario de Búsqueda** (registro, búsqueda, listado por departamento y eliminación), consultando además las tareas pendientes de cada empleado.
- Calcular el tiempo total estimado mediante **recursividad** y distribuir la carga de trabajo entre los empleados de cada departamento mediante **Divide y Vencerás**.
- Buscar una tarea por su ID con **cuatro métodos a la vez** (tabla hash, árbol binario de búsqueda, búsqueda binaria y búsqueda secuencial), mostrando una gráfica comparativa del número de comparaciones que realizó cada uno.
- Visualizar un **Árbol Binario de Búsqueda de tareas**, ejecutar sus **recorridos recursivos** (inorden, preorden y postorden) y **balancearlo** con un algoritmo de Divide y Vencerás.
- Ordenar las tareas por urgencia con **QuickSort**.
- Modelar dependencias entre tareas mediante un **grafo dirigido**, impidiendo ciclos desde el momento del registro y calculando el orden de ejecución con **ordenamiento topológico** (algoritmo de Kahn) agrupado por etapas.

La interfaz gráfica está redactada en lenguaje orientado al usuario final (por ejemplo, "Distribución de Tareas" o "Buscar Tarea por ID" en lugar de nombrar directamente los algoritmos); la terminología técnica de cada estructura se documenta en los comentarios del código fuente y en este documento. Como apoyo didáctico, algunos elementos sí muestran el nombre técnico cuando ayuda a entender el resultado (por ejemplo, la gráfica comparativa de búsquedas indica "Tabla hash:  $O(1)$ ", "Árbol (ABB):

$O(\log n)$ ", etc.). La interfaz cuenta además con modo oscuro, modo pantalla completa (F11 / Esc) y menú lateral colapsable.

La aplicación persiste la información de tareas activas y empleados en una base de datos SQLite, mediante una capa de persistencia dedicada (paquete persistencia, con las clases ConexionBD, TareaRepositorio y EmpleadoRepositorio).

Los datos se cargan al iniciar la aplicación y **se guardan de inmediato después de cada operación que modifica el estado** (y una vez más al cerrar la ventana). Las estructuras de datos en memoria (Pila, Cola, Lista, Cola de Prioridad, Árboles, Tabla Hash, Grafo) se reconstruyen a partir de la base de datos al iniciar y son las responsables de resolver toda la lógica y los algoritmos del sistema.

# Objetivos

## Objetivo general

Diseñar e implementar un sistema de gestión de tareas mediante estructuras de datos avanzadas y algoritmos de ordenamiento y búsqueda para optimizar las operaciones de asignación, consulta y organización de tareas de una empresa.

## Objetivos específicos

- **Implementar una Cola de Prioridad (Heap)** que gestione las tareas con base en su urgencia y fecha de entrega.
- **Implementar un Árbol Binario de Búsqueda (BST)** para organizar a los empleados por departamento y optimizar la búsqueda de asignaciones.
- **Aplicar recursividad** para calcular estadísticas de las tareas (tiempo total estimado y carga de trabajo por empleado) y para recorrer los árboles.
- **Aplicar la estrategia Divide y Vencerás** para distribuir las tareas entre los empleados de cada departamento y para balancear el árbol de tareas.
- **Emplear tablas hash (HashMap)** para almacenar y acceder rápidamente a la información de tareas y empleados.
- **Implementar un método de ordenamiento** (QuickSort) para organizar las tareas por urgencia.
- **Desarrollar y comparar métodos de búsqueda** (tabla hash, árbol binario de búsqueda, búsqueda binaria y búsqueda secuencial) para encontrar tareas específicas, evidenciando la diferencia de eficiencia entre ellos.
- **Utilizar un grafo dirigido** para representar las dependencias entre tareas, prevenir dependencias circulares y calcular su orden de ejecución por etapas.
- **Integrar todas las estructuras anteriores** bajo el patrón MVC, con persistencia en base de datos y una interfaz gráfica Swing clara y organizada por secciones de navegación.

## Estructura del Código

### Organización de paquetes

```
com.example
├─ GestionTareasApp.java          (Lanzador principal)
├─ Controlador/
│   └─ GestionTareasController.java (Lógica de control, listeners y
sincronización con la BD)
├─ Modelo/
│   ├─ Tarea.java
│   ├─ Empleado.java
│   ├─ PilaTareas.java
│   ├─ ColaTareas.java
│   ├─ ListaTareas.java
│   ├─ ColaPrioridadTareas.java
│   ├─ ArbolEmpleados.java
│   ├─ ArbolTareasABB.java      (NUEVO: ABB de tareas por ID, recorridos
y balanceo)
│   ├─ ProcesadorRecurativo.java
│   ├─ GestorTablasHashYAlgoritmos.java
│   └─ GrafoDependencias.java
├─ Vista/
│   ├─ GestionTareasView.java    (Interfaz gráfica Swing y componentes de
dibujo)
│   └─ SelectorFechaPanel.java   (Selector de fecha: calendario + texto
validado)
└─ persistencia/
    ├─ ConexionBD.java
    ├─ EmpleadoRepositorio.java
    └─ TareaRepositorio.java
```

Archivos adicionales del proyecto: demo/pom.xml (configuración Maven y dependencia de SQLite) y data/gestion\_tareas.db (base de datos generada automáticamente).

## Tabla de clases y responsabilidades

| Clase                      | Paquete     | Estructura de datos / algoritmo                            | Responsabilidad principal                                                           |
|----------------------------|-------------|------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>GestionTareasApp</b>    | com.example | —                                                          | Punto de entrada; instancia Modelo, Vista y Controlador y guarda en la BD al cerrar |
| <b>Tarea</b>               | Modelo      | —                                                          | Entidad que representa una tarea (incluye Responsable Directo y días restantes)     |
| <b>Empleado</b>            | Modelo      | —                                                          | Entidad que representa un empleado                                                  |
| <b>PilaTareas</b>          | Modelo      | Pila (java.util.Stack)                                     | Tareas urgentes (LIFO)                                                              |
| <b>ColaTareas</b>          | Modelo      | Cola (Queue / LinkedList)                                  | Tareas programadas (FIFO)                                                           |
| <b>ListaTareas</b>         | Modelo      | Lista (ArrayList)                                          | Tareas generales por departamento                                                   |
| <b>ColaPrioridadTareas</b> | Modelo      | Heap (PriorityQueue)                                       | Tareas ordenadas por urgencia y fecha de entrega                                    |
| <b>ArbolEmpleados</b>      | Modelo      | Árbol Binario de Búsqueda                                  | Registro, búsqueda, eliminación y consulta por departamento de empleados            |
| <b>ArbolTareasABB</b>      | Modelo      | Árbol Binario de Búsqueda / Recorridos / Divide y Vencerás | Búsqueda de tareas activas por ID, recorridos y balanceo del árbol                  |
| <b>ProcesadorRecursivo</b> | Modelo      | Recursividad / Divide y Vencerás                           | Cálculo de tiempos y distribución de carga por departamento                         |

|                                         |              |                                                                                     |                                                                                       |
|-----------------------------------------|--------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>GestorTablasHash<br/>YAlgoritmos</b> | Modelo       | HashMap /<br>QuickSort /<br>Búsqueda<br>binaria /<br>Búsqueda<br>secuencial         | Búsquedas O(1), ordenamiento<br>y comparativa de métodos de<br>búsqueda               |
| <b>GrafoDependencias</b>                | Modelo       | Grafo dirigido<br>(lista de<br>adyacencia) /<br>DFS / Orden<br>topológico<br>(Kahn) | Dependencias entre tareas,<br>prevención de ciclos y orden de<br>ejecución por etapas |
| <b>GestionTareasController</b>          | Controlador  | —                                                                                   | Conecta eventos de la Vista con<br>el Modelo y la persistencia                        |
| <b>GestionTareasView</b>                | Vista        | —                                                                                   | Interfaz gráfica (Swing), tablas,<br>tarjetas y visualizaciones                       |
| <b>SelectorFechaPanel</b>               | Vista        | —                                                                                   | Captura validada de la fecha de<br>entrega (texto o calendario)                       |
| <b>ConexionBD</b>                       | persistencia | —                                                                                   | Conexión JDBC a SQLite y<br>creación/migración de tablas                              |
| <b>TareaRepositorio</b>                 | persistencia | —                                                                                   | Guardar y cargar tareas de la BD                                                      |
| <b>EmpleadoRepositorio</b>              | persistencia | —                                                                                   | Guardar y cargar empleados de<br>la BD                                                |

## Fichas técnicas de las clases del sistema

A continuación, se describen las principales clases que conforman el sistema GestionTareasApp. Para cada clase se especifica el paquete al que pertenece, la estructura de datos o algoritmo utilizado y su responsabilidad dentro de la aplicación.

### GestionTareasApp

| Elemento                        | Descripción                                                                                                      |
|---------------------------------|------------------------------------------------------------------------------------------------------------------|
| Paquete                         | com.example                                                                                                      |
| Tipo                            | Clase principal                                                                                                  |
| Estructura de datos / algoritmo | No aplica                                                                                                        |
| Responsabilidad principal       | Constituye el punto de entrada de la aplicación y se encarga de iniciar los componentes principales del sistema. |

**Descripción:** La clase GestionTareasApp contiene el método principal de ejecución de la aplicación. Su función es inicializar el sistema, establecer la comunicación entre el Modelo, la Vista y el Controlador, y registrar un WindowListener que guarda la información en la base de datos cuando el usuario cierra la ventana.

### Tarea

| Elemento                        | Descripción                                                                   |
|---------------------------------|-------------------------------------------------------------------------------|
| Paquete                         | Modelo                                                                        |
| Tipo                            | Entidad                                                                       |
| Estructura de datos / algoritmo | No aplica                                                                     |
| Responsabilidad principal       | Representar la información y características de una tarea dentro del sistema. |

**Descripción:** La clase Tarea representa la unidad principal de información que administra la aplicación. Contiene los atributos necesarios para identificar y clasificar una tarea (ID, título, departamento, urgencia, tiempo estimado, fecha de entrega, estructura asignada) y, de forma opcional, su responsable directo. También calcula los días restantes para la fecha de entrega e indica si la tarea está vencida.

## Empleado

| Elemento                        | Descripción                                                     |
|---------------------------------|-----------------------------------------------------------------|
| Paquete                         | Modelo                                                          |
| Tipo                            | Entidad                                                         |
| Estructura de datos / algoritmo | No aplica                                                       |
| Responsabilidad principal       | Representar a los empleados asociados con la gestión de tareas. |

**Descripción:** La clase Empleado modela la información correspondiente a los empleados registrados en el sistema. Sus objetos se utilizan en el árbol binario de empleados, en la tabla hash de empleados, como responsable directo de las tareas y en la distribución de carga de trabajo.

## PilaTareas

| Elemento                        | Descripción                                        |
|---------------------------------|----------------------------------------------------|
| Paquete                         | Modelo                                             |
| Tipo                            | Estructura de datos                                |
| Estructura de datos / algoritmo | Pila (Stack)                                       |
| Principio de funcionamiento     | LIFO (Last In, First Out)                          |
| Responsabilidad principal       | Administrar las tareas clasificadas como urgentes. |

**Descripción:** La clase PilaTareas implementa una estructura de tipo pila para almacenar y procesar tareas urgentes. Su funcionamiento sigue el principio LIFO, mediante el cual la última tarea agregada es la primera en ser procesada. En la interfaz, la fila que corresponde al tope de la pila se señala con el indicador "TOPE: Siguiente".

## ColaTareas

| Elemento                        | Descripción                                                            |
|---------------------------------|------------------------------------------------------------------------|
| Paquete                         | Modelo                                                                 |
| Tipo                            | Estructura de datos                                                    |
| Estructura de datos / algoritmo | Cola (Queue / LinkedList)                                              |
| Principio de funcionamiento     | FIFO (First In, First Out)                                             |
| Responsabilidad principal       | Administrar las tareas programadas de acuerdo con su orden de llegada. |

**Descripción:** La clase ColaTareas utiliza una estructura de tipo cola para almacenar las tareas programadas. Su funcionamiento se basa en el principio FIFO, por lo que la primera tarea que ingresa es la primera que será procesada. En la interfaz, la fila del frente de la cola se señala con el indicador "FRENTE: Siguiente".

## ListaTareas

| Elemento                        | Descripción                                                            |
|---------------------------------|------------------------------------------------------------------------|
| Paquete                         | Modelo                                                                 |
| Tipo                            | Estructura de datos                                                    |
| Estructura de datos / algoritmo | Lista (ArrayList)                                                      |
| Responsabilidad principal       | Administrar tareas generales y facilitar su consulta por departamento. |

**Descripción:** La clase ListaTareas administra un conjunto de tareas generales mediante una lista dinámica. Permite almacenar, recorrer, filtrar por departamento y eliminar tareas mediante su identificador. Sobre una copia de esta lista se aplica el ordenamiento QuickSort por urgencia.

### ColaPrioridadTareas

| Elemento                        | Descripción                                                                    |
|---------------------------------|--------------------------------------------------------------------------------|
| Paquete                         | Modelo                                                                         |
| Tipo                            | Estructura de datos                                                            |
| Estructura de datos / algoritmo | Cola de prioridad (PriorityQueue, implementada internamente como heap binario) |
| Principio de funcionamiento     | Priorización de elementos                                                      |
| Responsabilidad principal       | Organizar las tareas de acuerdo con su nivel de urgencia y fecha de entrega.   |

**Descripción:** La clase ColaPrioridadTareas utiliza una cola de prioridad para organizar las tareas. A diferencia de una cola convencional, los elementos no se procesan según su orden de llegada, sino considerando primero su urgencia y, en caso de empate, la fecha de entrega más próxima.

### ArbolEmpleados

| Elemento                        | Descripción                                                          |
|---------------------------------|----------------------------------------------------------------------|
| Paquete                         | Modelo                                                               |
| Tipo                            | Estructura de datos                                                  |
| Estructura de datos / algoritmo | Árbol Binario de Búsqueda                                            |
| Responsabilidad principal       | Registrar, organizar, consultar y eliminar información de empleados. |

**Descripción:** La clase ArbolEmpleados implementa un Árbol Binario de Búsqueda que organiza a los empleados usando su ID como clave. Permite insertar, buscar y eliminar empleados de forma recursiva, así como obtener los empleados de un departamento mediante un recorrido inorden. Esta última operación se utiliza en varios puntos del sistema: el listado por departamento, el selector de responsable directo y la distribución de tareas.

## ArbolTareasABB

| Elemento                        | Descripción                                                                                                                        |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Paquete                         | Modelo                                                                                                                             |
| Tipo                            | Estructura de datos                                                                                                                |
| Estructura de datos / algoritmo | Árbol Binario de Búsqueda por ID; recorridos recursivos; balanceo por Divide y Vencerás                                            |
| Responsabilidad principal       | Mantener las tareas activas organizadas por ID para buscarlas eficientemente y demostrar los recorridos y el balanceo de un árbol. |

**Descripción:** La clase ArbolTareasABB organiza las tareas activas en un Árbol Binario de Búsqueda cuya clave es el ID (folio) de la tarea. Implementa inserción, búsqueda y eliminación recursivas, los recorridos inorden, preorden y postorden, el cálculo de la altura del árbol y un algoritmo de balanceo que reconstruye el árbol con altura mínima. También ofrece una búsqueda que cuenta las comparaciones realizadas y guarda la ruta de nodos visitados, información que la Vista usa para la gráfica comparativa y para resaltar el camino dentro del dibujo del árbol.

## ProcesadorRecursivo

| Elemento                        | Descripción                                                                                                  |
|---------------------------------|--------------------------------------------------------------------------------------------------------------|
| Paquete                         | Modelo                                                                                                       |
| Tipo                            | Componente algorítmico                                                                                       |
| Estructura de datos / algoritmo | Recursividad / Divide y Vencerás                                                                             |
| Responsabilidad principal       | Calcular el tiempo total estimado y distribuir la carga de trabajo entre los empleados de cada departamento. |

**Descripción:** La clase ProcesadorRecursivo concentra operaciones que utilizan técnicas de recursividad. Calcula el tiempo total estimado de las tareas mediante una suma recursiva y distribuye las tareas sin responsable entre los empleados de su mismo departamento aplicando Divide y Vencerás, asignando cada tarea al empleado con menor carga de horas acumuladas (carga que también se calcula de forma recursiva).

## GestorTablasHashYAlgoritmos

| Elemento                        | Descripción                                                                                              |
|---------------------------------|----------------------------------------------------------------------------------------------------------|
| Paquete                         | Modelo                                                                                                   |
| Tipo                            | Componente algorítmico                                                                                   |
| Estructura de datos / algoritmo | HashMap, QuickSort, búsqueda binaria y búsqueda secuencial                                               |
| Responsabilidad principal       | Realizar operaciones eficientes de búsqueda y ordenamiento, y medir el costo de cada método de búsqueda. |

**Descripción:** La clase GestorTablasHashYAlgoritmos concentra diferentes estructuras y algoritmos utilizados para optimizar el manejo de información. El uso de HashMap permite acceder a tareas y empleados mediante su clave en tiempo promedio constante; QuickSort ordena las tareas por urgencia; y la búsqueda binaria localiza tareas dentro de una colección ordenada por ID. Además, incluye versiones de las búsquedas que cuentan las comparaciones realizadas, incluida una búsqueda secuencial de referencia, para comparar su eficiencia dentro de la interfaz.

## GrafoDependencias

| Elemento                        | Descripción                                                                                                  |
|---------------------------------|--------------------------------------------------------------------------------------------------------------|
| Paquete                         | Modelo                                                                                                       |
| Tipo                            | Estructura de datos                                                                                          |
| Estructura de datos / algoritmo | Grafo dirigido mediante lista de adyacencia; DFS recursivo; ordenamiento topológico (Kahn)                   |
| Responsabilidad principal       | Representar las dependencias entre tareas activas, impedir ciclos y determinar un orden de ejecución válido. |

**Descripción:** La clase GrafoDependencias representa las relaciones de dependencia entre tareas mediante un grafo dirigido en el que cada nodo es el ID de una tarea y cada arista previa → siguiente indica que la tarea previa debe terminarse antes que la siguiente. Antes de agregar una arista valida que no se trate de la misma tarea, que no exista ya, que ambas tareas pertenezcan al mismo departamento y que no se forme un ciclo (mediante una búsqueda en profundidad). Gracias a ello, el grafo siempre es

acíclico y siempre existe un orden de ejecución, que se calcula por etapas con el algoritmo de Kahn.

### **GestionTareasController**

| Elemento                        | Descripción                                                                                                               |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Paquete                         | Controlador                                                                                                               |
| Tipo                            | Controlador                                                                                                               |
| Estructura de datos / algoritmo | No aplica (orquesta todas las estructuras del Modelo)                                                                     |
| Responsabilidad principal       | Gestionar los eventos de la interfaz, comunicar la Vista con el Modelo y sincronizar la información con la base de datos. |

**Descripción:** La clase GestionTareasController funciona como intermediaria entre la interfaz gráfica, la lógica de la aplicación y la capa de persistencia. Recibe las acciones realizadas por el usuario en la Vista, ejecuta las operaciones correspondientes sobre las estructuras del Modelo, mantiene sincronizadas las estructuras entre sí (por ejemplo, el árbol de tareas y el grafo con las tareas activas), actualiza la información presentada y guarda los cambios en la base de datos.

### **GestionTareasView**

| Elemento                  | Descripción                                                                     |
|---------------------------|---------------------------------------------------------------------------------|
| Paquete                   | Vista                                                                           |
| Tipo                      | Interfaz gráfica                                                                |
| Tecnología                | Java Swing (con componentes de dibujo personalizados mediante Graphics2D)       |
| Responsabilidad principal | Proporcionar la interfaz mediante la cual el usuario interactúa con el sistema. |

**Descripción:** La clase GestionTareasView constituye la interfaz gráfica de la aplicación. Contiene los paneles, formularios, tablas, botones y visualizaciones (gráficas, dibujo del árbol y del grafo) necesarios para que el usuario pueda registrar, consultar y procesar tareas. La Vista se comunica con el Controlador sin incorporar la lógica de procesamiento de las estructuras de datos.

### SelectorFechaPanel

| Elemento                  | Descripción                                                   |
|---------------------------|---------------------------------------------------------------|
| Paquete                   | Vista                                                         |
| Tipo                      | Componente gráfico                                            |
| Tecnología                | Java Swing (JTextField + calendario emergente con JPopupMenu) |
| Responsabilidad principal | Capturar y validar la fecha de entrega de una tarea.          |

**Descripción:** Componente compuesto que permite escribir la fecha de entrega o seleccionarla en un mini-calendario. Valida la fecha de forma estricta contra varios formatos admitidos y entrega al Controlador un `LocalDate` ya validado.

### ConexionBD, TareaRepositorio y EmpleadoRepositorio

| Elemento                  | Descripción                                                  |
|---------------------------|--------------------------------------------------------------|
| Paquete                   | persistencia                                                 |
| Tipo                      | Acceso a datos (DAO / Repositorio)                           |
| Tecnología                | JDBC + SQLite                                                |
| Responsabilidad principal | Crear la base de datos, guardar y cargar tareas y empleados. |

**Descripción:** ConexionBD crea la carpeta data/ y las tablas tareas y empleados si no existen, y aplica una migración que agrega la columna `responsable_id` a bases de datos creadas con versiones anteriores. TareaRepositorio y EmpleadoRepositorio guardan la información dentro de una transacción (si ocurre un error se hace *rollback*) y la cargan al iniciar la aplicación.

## Relación de las clases con el patrón MVC

Las clases principales del sistema se organizan siguiendo el patrón Modelo-Vista-Controlador (MVC), complementado con una capa de persistencia, permitiendo separar las responsabilidades de los diferentes componentes de la aplicación.

| Componente          | Clases principales                                                                                                                                                             | Función                                                                                                  |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Modelo</b>       | Tarea, Empleado, PilaTareas, ColaTareas, ListaTareas, ColaPrioridadTareas, ArbolEmpleados, ArbolTareasABB, ProcesadorRecursivo, GestorTablasHashYAlgoritmos, GrafoDependencias | Contiene las entidades, estructuras de datos y algoritmos que conforman la lógica del sistema.           |
| <b>Vista</b>        | GestionTareasView, SelectorFechaPanel                                                                                                                                          | Presenta la información y proporciona los elementos gráficos para la interacción con el usuario.         |
| <b>Controlador</b>  | GestionTareasController                                                                                                                                                        | Procesa las acciones del usuario y coordina la comunicación entre la Vista, el Modelo y la persistencia. |
| <b>Persistencia</b> | ConexionBD, TareaRepositorio, EmpleadoRepositorio                                                                                                                              | Almacena y recupera tareas y empleados en la base de datos SQLite.                                       |
| <b>Aplicación</b>   | GestionTareasApp                                                                                                                                                               | Inicializa y pone en funcionamiento la aplicación.                                                       |

### Consideraciones

La separación de estas responsabilidades facilita el mantenimiento, comprensión y ampliación del sistema. Las estructuras de datos se encuentran concentradas en el Modelo, la Vista se encarga de la presentación, el Controlador de la interacción entre

componentes y la capa de persistencia del almacenamiento. De esta manera, las estructuras de datos y algoritmos pueden utilizarse sin depender de los elementos gráficos ni de la base de datos.

## Explicación Detallada de Cada Componente

### Tarea (Modelo)

Entidad principal del sistema. Atributos: id, titulo, departamento, urgencia (escala de 1 a 5), tipoEstructura (texto que indica a qué estructura pertenece), tiempoEstimado (horas), fechaEntrega (LocalDate) y responsableDirecto (Empleado, opcional; null significa "Sin Asignar").

Constructores:

- **Constructor completo:** recibe todos los datos; el ID se autogenera mediante el contador estático contadorId. Si la fecha es null, se asigna la fecha actual (LocalDate.now()).
- **Constructor con ID explícito:** se usa al cargar desde la base de datos; conserva el ID guardado y ajusta el contador con  $\text{contadorId} = \max(\text{contadorId}, \text{id} + 1)$ , de modo que las nuevas tareas nunca repitan un ID existente.
- **Constructor con Responsable Directo:** igual al completo, pero además asigna el responsable validando la regla de negocio.
- **Constructor de compatibilidad:** fija tiempoEstimado = 2 y fechaEntrega = mañana.

Métodos relevantes:

- **setResponsableDirecto(Empleado):** aplica la regla de negocio: si el empleado no pertenece al mismo departamento que la tarea, lanza IllegalArgumentException. Con null la tarea queda "Sin Asignar".
- **getNombreResponsable():** devuelve el nombre del responsable o "Sin Asignar".
- **getDiasRestantes() / estaVencida():** calculan los días que faltan para la fecha de entrega (negativo si ya venció) con ChronoUnit.DAYS.between.
- **describirFechaEntrega(LocalDate):** genera textos como "28/09/2026 (4 días restantes)", "24/09/2026 (vence hoy)" o "20/09/2026 (vencida hace 4 días)".
- **toString():** da formato de una sola línea con todos los campos, usado en los diálogos de la aplicación.

## Empleado (Modelo)

Clase simple sin setters: id (String; se compara lexicográficamente, no numéricamente), nombre y departamento. Su toString() devuelve "Nombre (ID)", texto que se muestra en el selector de Responsable Directo. Se usa en el árbol binario de empleados, en la tabla hash de empleados y como responsable de las tareas.

## PilaTareas (Estructura: Pila / LIFO)

Envuelve un java.util.Stack<Tarea>. Expone:

- **push(Tarea):** agrega una tarea urgente a la cima.
- **pop():** retira y retorna la tarea de la cima (o null e imprime aviso en consola si está vacía).
- **peek():** consulta la tarea de la cima sin retirarla.
- **getPila():** expone la estructura completa para volcarla en la tabla de la Vista.

Cada operación imprime trazas en la consola de Java con el prefijo [CONSOLA - PILA]. En la tabla, la columna "Turno" marca con una flecha y el texto "TOPE: Siguiendo" la tarea que se procesará con el siguiente *pop* (la última fila, que es la última insertada).

## ColaTareas (Estructura: Cola / FIFO)

Envuelve un Queue<Tarea> implementado con LinkedList. Expone enqueue(), dequeue() y front(), con orden de llegada (FIFO). Registra trazas con el prefijo [CONSOLA - COLA]. En la tabla, la columna "Turno" marca con "FRENTE: Siguiendo" la primera tarea insertada.

## ListaTareas (Estructura: Lista dinámica)

Envuelve un ArrayList<Tarea> para las tareas "generales" clasificadas por departamento. Expone:

- **insert(Tarea):** agrega al final de la lista.
- **delete(int id):** recorre linealmente la lista y elimina la tarea cuyo ID coincide; retorna true/false según el resultado. El Controlador usa ese valor para informar al usuario si el ID no existía.
- **find(String departamento):** filtra (comparación insensible a mayúsculas) y retorna todas las tareas de un departamento.
- **getLista():** expone la lista completa (usada también para el QuickSort).

### **ColaPrioridadTareas (Estructura: Heap / Cola de Prioridad)**

Envuelve un PriorityQueue<Tarea> con un Comparator personalizado: ordena primero por mayor urgencia y, en caso de empate, por fecha de entrega más cercana. Expone agregarTarea(), extraerSiguienteTarea() (*poll*), verSiguienteTarea() (*peek*) y estaVacia().

Además incluye obtenerTareasOrdenadas(), que devuelve una copia de las tareas ya ordenadas según el mismo criterio, sin alterar la cola real. Este método permite al Controlador refrescar la tabla de esta estructura y, desde esta versión, incluir las tareas de la Cola de Prioridad en el consolidado general de tareas activas.

### **ArbolEmpleados (Estructura: Árbol Binario de Búsqueda - BST)**

Implementa un BST con una clase interna privada Nodo (empleado + referencias izquierdo/derecho). Los empleados se ordenan por su id mediante compareTo (orden lexicográfico, no numérico; por ejemplo, "10" se ubica antes que "9").

#### **Aclaración sobre la organización de empleados por departamento**

El Árbol Binario de Búsqueda utiliza el ID del empleado como clave de ordenamiento, de modo que la inserción, la búsqueda directa y la eliminación se realizan sobre una estructura jerárquica organizada por identificador. Para consultar y organizar a los empleados por departamento, el sistema realiza un recorrido inorden del árbol y aplica un filtro sobre el atributo departamento. Como el recorrido inorden visita los nodos en orden ascendente de ID, los empleados de un departamento se obtienen también ordenados por ID. Por lo tanto, el departamento no es la clave estructural del BST, sino el criterio de filtrado durante el recorrido; esta decisión conserva la eficiencia de la búsqueda individual por ID y, al mismo tiempo, proporciona la consulta por departamento que utilizan el listado de empleados, el selector de responsable directo y la distribución de tareas.

- **insertar(Empleado) / insertarRec(...):** inserción recursiva estándar de BST (un ID repetido no se inserta en el árbol).
- **buscarPorId(String) / buscarRec(...):** búsqueda recursiva,  $O(\log n)$  en árboles balanceados y  $O(n)$  en el peor caso si el árbol degenera.
- **eliminar(String) / eliminarRec(...) (nuevo):** eliminación recursiva que contempla los tres casos clásicos: nodo sin hijos, nodo con un solo hijo (se "sube" el hijo) y nodo con dos hijos (se reemplaza por su sucesor inorden, el mínimo del subárbol derecho). Devuelve el empleado eliminado o null si no existía.

- **obtenerPorDepartamento(String) / buscarPorDeptRec(...):** recorrido inorden (izquierda → nodo → derecha) filtrando por departamento.

### **ArbolTareasABB (Estructura: Árbol Binario de Búsqueda de tareas)**

Árbol Binario de Búsqueda cuya clave es el ID de la tarea. Contiene una clase interna pública `Nodo` de solo lectura (para que la Vista pueda dibujarlo) y el *record* `ResultadoBusquedaABB(tarea, comparaciones, ruta)`. El Controlador lo mantiene sincronizado con las tareas activas: en cada actualización inserta las tareas nuevas y elimina las que ya fueron procesadas (`sincronizarArbolTareas()`).

- **insertar(Tarea) / insertarRec(...):** inserción recursiva; si el ID ya existe no se duplica y el método devuelve false.
- **buscar(int id):** búsqueda recursiva que desciende a la izquierda si el ID buscado es menor y a la derecha si es mayor. Costo  $O(h)$ , donde  $h$  es la altura del árbol.
- **buscarConConteo(int id):** misma búsqueda, pero registra cada nodo visitado; el número de nodos visitados es el número de comparaciones y la lista de IDs es la ruta que se resalta en el dibujo.
- **eliminar(int id) / eliminarRec(...):** eliminación recursiva con los tres casos (sin hijos, un hijo, dos hijos con sucesor inorden).
- **inorden(), preorden(), postorden():** recorridos recursivos. Inorden (Izquierda–Raíz–Derecha) devuelve los IDs ordenados de menor a mayor; preorden (Raíz–Izquierda–Derecha) comienza siempre por la raíz y sirve para copiar o reconstruir el árbol con la misma forma; postorden (Izquierda–Derecha–Raíz) visita la raíz al final y es el orden que se usa para eliminar un árbol desde las hojas.
- **balancear():** algoritmo de **Divide y Vencerás**. Primero obtiene las tareas en orden (recorrido inorden); después `construirBalanceado(lista, inicio, fin)` toma el elemento central como raíz (**divide**), construye recursivamente el subárbol izquierdo con la mitad izquierda y el derecho con la mitad derecha (**vence**) y une ambos subárboles al nodo (**combina**). Caso base: sublista vacía ( $\text{inicio} > \text{fin}$ ). Devuelve la altura antes y después del balanceo.
- **altura() / alturaMinima():** la altura se calcula recursivamente (árbol vacío = 0); la altura mínima posible para  $n$  nodos es  $\lceil \log_2(n + 1) \rceil$ . La Vista compara ambos valores para indicar si el árbol está desbalanceado.

Nota: como los IDs de las tareas son consecutivos y el Controlador inserta las tareas en orden de ID, el árbol tiende a degenerar en una "lista" inclinada a la derecha (altura = n). Precisamente esta situación permite demostrar el efecto del balanceo: tras presionar "Balancear árbol", la altura baja a la mínima posible y las búsquedas requieren menos comparaciones.

### **ProcesadorRecursoivo (Rekursividad / Divide y Vencerás)**

- **calcularTiempoTotalEstimado(List<Tarea>, int index):** recursión simple de acumulación. Caso base: `index >= tareas.size()` retorna 0. Caso recursivo: suma el tiempo de la tarea actual más el resultado de la llamada con `index + 1`. Se aplica sobre todas las tareas activas (Pila, Cola, Lista y Cola de Prioridad).
- **distribuirTareasDivideYVencerás(List<Tarea>, ArbolEmpleados):** coordina la distribución en tres pasos:
  1. Para cada departamento que tiene tareas, obtiene su personal desde el árbol de empleados (`obtenerPorDepartamento`) e inicializa una lista vacía de asignaciones por empleado.
  2. Las tareas que ya tienen Responsable Directo se respetan y se cuentan como carga de ese empleado; solo las tareas "Sin Asignar" se agrupan por departamento para repartirse.
  3. Para cada departamento con personal, invoca `distribuirRec()` sobre sus tareas sin asignar. Las tareas de departamentos sin personal quedan sin asignar (el Controlador las reporta con una alerta).
- **distribuirRec(tareas, inicio, fin, empleados, asignaciones):** esquema Divide y Vencerás. **Divide:** parte el rango `[inicio, fin]` a la mitad. **Vence:** resuelve recursivamente cada mitad. **Caso base:** cuando el subrango contiene una sola tarea (`inicio == fin`), la asigna al empleado con menor carga.
- **obtenerEmpleadoConMenosCargaPorTiempo(...):** recorre los empleados del departamento y elige al que tiene **menos horas acumuladas** (no menos tareas), logrando un balanceo por tiempo de trabajo.
- **calcularCarga(...) / calcularCargaRekursiva(...):** suma recursivamente el tiempo estimado de las tareas asignadas a un empleado.

## GestorTablasHashYAlgoritmos (Tablas Hash / QuickSort / Búsquedas)

- **mapaTareas: Map<Integer, Tarea> y mapaEmpleados: Map<String, Empleado>**: dos tablas hash (HashMap) independientes.
- **guardarTarea(Tarea) / buscarTareaPorHash(int id)**: inserción y búsqueda en tiempo promedio  $O(1)$ . Toda tarea creada o cargada durante la sesión se guarda aquí, sin importar su estructura; por ello la tabla hash conserva también las tareas que ya fueron procesadas en la sesión actual, lo que permite al sistema distinguir entre una tarea "ya atendida" y una tarea inexistente.
- **guardarEmpleado(Empleado) / buscarEmpleadoPorHash(String id) / eliminarEmpleado(String id)**: operaciones análogas para empleados (la eliminación es nueva y se usa al dar de baja a un empleado).
- **ordenarTareasPorUrgenciaQuickSort(List<Tarea>, izq, der) / particionar(...)**: QuickSort con partición de Lomuto (pivote = último elemento); el resultado queda ordenado de mayor a menor urgencia.
- **busquedaBinariaPorId(List<Tarea>, idBuscado)**: búsqueda binaria iterativa,  $O(\log n)$ ; requiere que la lista esté ordenada por ID.
- **Búsquedas con conteo de comparaciones (nuevas)**: se definen con el *record* ResultadoBusqueda(tarea, comparaciones):
  - **buscarPorHashConConteo(id)**: acceso directo a la casilla calculada por el hash ( $\approx 1$  comparación).
  - **busquedaBinariaConConteo(lista, id)**: cuenta una comparación por cada vez que se examina el elemento central.
  - **busquedaSecuencialConConteo(lista, id)**: revisa los elementos uno por uno; sirve como referencia  $O(n)$  para contrastar la eficiencia de los demás métodos.

## GrafoDependencias (Grafo dirigido / DFS / Orden topológico)

Modela las dependencias usando como nodos los IDs de las tareas (int), una lista de adyacencia Map<Integer, Set<Integer>> y un mapa de grados de entrada Map<Integer, Integer> (ambos TreeMap/TreeSet para que la salida sea ordenada y estable). Una arista previa → siguiente significa "la tarea previa debe terminarse antes que la siguiente".

- **agregarTarea(int id):** registra un nodo (idempotente, usa putIfAbsent).
- **agregarDependencia(Tarea previa, Tarea siguiente):** aplica la **regla de negocio de mismo departamento**; si los departamentos son distintos lanza DependenciaEntreDepartamentosException ("Transición no permitida"). Si coinciden, delega en la versión por ID.
- **agregarDependencia(int previa, int siguiente):** valida y rechaza con IllegalArgumentException: (1) una tarea que depende de sí misma, (2) una dependencia duplicada y (3) una dependencia que formaría un ciclo. Si todo es válido, crea la arista e incrementa el grado de entrada del destino.
- **existeCamino(origen, destino) / existeCaminoRec(...):** búsqueda en profundidad (DFS) recursiva con conjunto de visitados. Antes de agregar previa → siguiente se verifica si ya existe un camino de siguiente a previa; si existe, la nueva arista cerraría un ciclo y se bloquea. Con esto el grafo es siempre un **grafo acíclico dirigido (DAG)**.
- **eliminarTarea(int id):** retira un nodo y todas sus aristas, ajustando los grados de entrada. El Controlador lo usa automáticamente cuando una tarea sale de su estructura (se procesa o elimina).
- **limpiar():** elimina todas las dependencias.
- **calcularOrdenEjecucion():** algoritmo de **Kahn** para ordenamiento topológico. Coloca en una cola de prioridad (PriorityQueue) los nodos con grado de entrada 0; al procesar cada nodo, descuenta el grado de entrada de sus vecinos y agrega a la cola los que llegan a 0. Entre varias tareas disponibles se elige el ID menor, para que el resultado sea estable. Además calcula la **etapa** de cada tarea (1 + la mayor etapa de sus requisitos): las tareas de una misma etapa no dependen entre sí y podrían realizarse en paralelo. Si el número de nodos procesados no coincide con el total, lanza IllegalStateException (protección defensiva; en la

práctica no ocurre porque los ciclos se bloquean al registrar). Devuelve el *record* OrdenEjecucion(secuencia, etapas).

### **GestionTareasController (Controlador)**

Recibe en su constructor la Vista y las tres estructuras base (Pila, Cola, Lista), e internamente instancia las estructuras avanzadas (ColaPrioridadTareas, ArbolEmpleados, GestorTablasHashYAlgoritmos, GrafoDependencias, ArbolTareasABB) y una lista auxiliar de empleados (listaEmpleadosMemoria) que conserva el orden de registro para mostrarlos en la tabla. Posteriormente inicializa la persistencia, registra los listeners y refresca la interfaz.

Responsabilidades principales:

- **Persistencia:**
  - inicializarPersistencia() crea la conexión y los repositorios y llama a cargarDesdeBD(). Si la carga falla, muestra un aviso y desactiva el guardado (persistenciaActiva = false) para no sobrescribir la base de datos con información incompleta.
  - cargarDesdeBD() carga primero los empleados (en el árbol, la tabla hash y la lista) y después las tareas, resolviendo su Responsable Directo con el árbol de empleados y colocando cada tarea en la estructura indicada por su tipoEstructura.
  - guardarCambios() se ejecuta al final de actualizarTablasYMetricas(), es decir, tras cada operación que modifica datos; guardarEnBD() se ejecuta al cerrar la ventana. Si el guardado falla, se muestra un único aviso por racha de errores.
- **initControlador():** registra todos los ActionListener de los botones y los listeners de cambio (departamento del formulario, selección de empleado y tarea previa del grafo).
- **agregarTarea():** valida el título y la fecha de entrega (vista.getSelectorFecha().obtenerFechaValidada()); valida que el Responsable Directo pertenezca al departamento; crea la Tarea, la guarda en la tabla hash y la coloca en la Pila, la Cola de Prioridad, la Cola secuencial o la Lista general según la estructura elegida.

- **refrescarResponsablesDisponibles():** al cambiar el departamento del formulario, llena el selector de Responsable Directo solo con los empleados de ese departamento (recorrido del árbol de empleados).
- **Métodos por estructura:** procesarPopPila, consultarPeekPila, procesarDequeCola, consultarFrontCola, eliminarDeLista (distingue "ID no numérico" de "ID inexistente"), buscarPorDepartamento, ordenarQuickSort, extraerColaPrioridad, verColaPrioridad.
- **Empleados:** agregarEmpleado, buscarEmpleadoBST, listarEmpleadosDeptoBST, mostrarTodosLosEmpleados y eliminarEmpleado (nuevo). Al eliminar un empleado (por el ID escrito o la fila seleccionada), se pide confirmación, se retira del árbol, de la tabla hash y de la lista, y sus tareas activas quedan "Sin Asignar"; si su departamento se queda sin personal, se avisa al usuario.
- **Tareas pendientes por empleado (nuevo):** obtenerPendientesPorEmpleado() recorre todas las estructuras activas y agrupa las tareas por responsable, ordenadas por fecha de entrega más próxima y luego por mayor urgencia. La tabla de empleados muestra el total de pendientes y, al seleccionar un empleado, un panel inferior muestra el detalle de sus tareas.
- **Distribución:** calcularTiempoRecurso() muestra el total de tareas y horas en tarjetas de métricas; distribuirDivideVenceras() ejecuta el reparto, **asigna de forma permanente** el responsable a las tareas que estaban "Sin Asignar", identifica los departamentos con tareas pero sin personal (alerta roja) y presenta el resultado agrupado por empleado, indicando si cada asignación es "Directa" (ya tenía responsable) o "Nueva".
- **Búsquedas (módulo rediseñado):**
  - buscarPorFolio() busca un ID con los cuatro métodos (tabla hash, ABB, búsqueda binaria sobre las tareas activas ordenadas por ID y búsqueda secuencial), muestra la tarjeta de la tarea encontrada con su estado ("Activa" o "Ya atendida"), la gráfica comparativa de comparaciones y resalta la ruta de búsqueda en el dibujo del árbol.
  - mostrarRecorrido(tipo) ejecuta el recorrido inorden, preorden o postorden y muestra la secuencia con una explicación.
  - balancearArbol() balancea el ABB y actualiza su visualización.

- sincronizarArbolTareas() y refrescarArbolBusquedas() mantienen el ABB igual a las tareas activas e informan altura actual, altura mínima y si el árbol está desbalanceado.
- **Grafo:** validarTareaActiva() acepta textos como "#5 · Backup BD", "#5" o "5" y distingue entre tarea finalizada y tarea inexistente; agregarDependenciaGrafo() muestra un mensaje específico para cada regla violada; filtrarTareasSiguienesPorDepartamento() limita las opciones de la tarea siguiente al departamento de la tarea previa; refrescarGrafo() retira del grafo las tareas finalizadas y redibuja; calcularOrdenTopologico() y limpiarGrafo().
- **obtenerTodasLasTareas():** consolida en una sola lista las tareas activas de **Pila + Cola + Lista + Cola de Prioridad** (en la versión anterior no incluía la Cola de Prioridad).
- **actualizarTablasYMetricas():** refresca las tablas de Pila, Cola, Lista, Cola de Prioridad y "Todas"; calcula horas y número de tareas por departamento para el Menú Principal; actualiza la tabla de empleados, el árbol de búsquedas y el grafo; y guarda los cambios en la base de datos.

### GestionTareasView (Vista)

Extiende JFrame y construye toda la interfaz de forma programática. Usa:

- **CardLayout** para alternar entre 11 secciones: Menú Principal, Registrar Tarea, Pilas (Urgentes), Colas (Programadas), Listas (Generales), Cola Prioridad, Empleados, Distribución de Tareas, Búsquedas, Grafo Dependencias y Ver Todas / Consola.
- **Cabecera** con el botón "|||" para colapsar/expandir el menú lateral y el botón "Pantalla Completa / Ventana Flotante" (atajos F11 y Esc).
- **Menú lateral colapsable** (alternarSidebar()), que muestra abreviaturas cuando está contraído.
- **Menú Principal (dashboard):** tres tarjetas con pendientes y resueltas de Pila, Cola y Lista; mini-tarjetas con las horas estimadas por departamento; y una gráfica de barras (DashboardChartPanel) con el número de tareas por departamento.
- **Formulario de registro:** título, departamento, responsable directo (filtrado por departamento, con la opción "Sin Asignar (se asignará en Distribución)"),

urgencia, tiempo estimado, fecha de entrega (SelectorFechaPanel) y estructura destino.

- **Tablas** (JTable + DefaultTableModel, no editables) con columnas comunes: ID, Título, Departamento, responsable directo, Urgencia, Tiempo (hrs) y "Fecha de Entrega y Días Restantes". Se apoyan en *renderers* personalizados: TurnoRenderer (indicador TOPE/FRENTE en Pila y Cola), FechaEntregaRenderer (fecha con días restantes, en rojo si está vencida) y ResponsableRenderer ("Sin Asignar" en cursiva y gris).
- **Sección Empleados:** formulario de registro, acciones (Buscar por ID, Eliminar por ID, Listar por Depto, Ver Todos), tabla de empleados con total de tareas pendientes y panel desplegable con el detalle de pendientes del empleado seleccionado.
- **Sección Distribución de Tareas:** tarjetas de métricas (KpiCard), alerta roja de departamentos sin personal (AlertaPanel) y un bloque por empleado con su encabezado (EncabezadoEmpleado: nombre, ID, departamento, número de tareas y horas de carga) y su tabla de tareas asignadas.
- **Sección Búsquedas:** buscador por ID (Enter o botón "Buscar"), tarjeta de detalle (DetalleTareaCard), gráfica de barras de comparaciones por método (GraficaComparaciones), selector de recorrido (Inorden, Preorden, Postorden) con la secuencia numerada (SecuenciaFolios) y el visualizador del árbol (ArbolVisual), que dibuja cada nodo según su posición inorden y su profundidad, resalta la ruta de búsqueda, marca en morado el nodo encontrado y numera el orden de visita del recorrido elegido.
- **Sección Grafo:** formulario con selectores editables de tarea previa y tarea siguiente (solo tareas activas), panel del orden de ejecución por etapas (OrdenEjecucionPanel) y dibujo del grafo (GrafoVisual), en el que cada columna es una etapa y las flechas van de la tarea previa a la dependiente; cada nodo muestra su número de paso.
- **Sección Ver Todas / Consola:** consolidado general de tareas activas, botón "Ver Todas (Ordenadas por Urgencia/Depto)", botón "Cambiar a modo oscuro" y la consola de eventos en tiempo real (logGUI).
- **Modo oscuro:** paleta completa de colores alternativa que se aplica a toda la interfaz, incluidas las visualizaciones personalizadas.

- **Métodos fábrica de estilo** (crearBotonEstilizado, crearBordeSeccion, crearTablaEstilizada, estilarCampoTexto) para mantener consistencia visual.
- **Getters, setters y métodos de presentación** (mostrarResultadoBusqueda, mostrarDistribucion, mostrarGrafo, actualizarArbol, actualizarDashboard, etc.) para que el Controlador registre listeners y envíe datos sin que la Vista conozca la lógica de negocio. La Vista intercambia datos con el Controlador mediante *records* simples: ResumenPendientes, FilaAsignacion, GrupoDistribucion, MetodoBusqueda y OpcionTarea.

### **SelectorFechaPanel (Vista)**

Componente compuesto (JPanel) que combina un JTextField y un botón "Elegir Fecha" que despliega un mini-calendario emergente construido con JPopupMenu y una cuadrícula de botones por día, con navegación entre meses ("<" / ">"). Ambas vías de entrada convergen en el mismo campo de texto, que se rellena con el formato yyyy-MM-dd.

La validación estricta (DateTimeFormatter con ResolverStyle.STRICT, que además rechaza fechas inexistentes como 2025-02-30) se ejecuta bajo demanda mediante obtenerFechaValidada(), que prueba en orden los formatos admitidos: yyyy-MM-dd, dd/MM/yyyy, dd-MM-yyyy, yyyy/MM/dd y MM/dd/yyyy. Retorna null si el campo está vacío (el Controlador lo interpreta como "usar la fecha de hoy") o lanza DateTimeParseException si el texto no coincide con ningún formato. Como dd/MM/yyyy se prueba antes que MM/dd/yyyy, una fecha ambigua como 03/04/2026 se interpreta como 3 de abril.

### Capa de persistencia (ConexionBD, TareaRepositorio, EmpleadoRepositorio)

- **ConexionBD:** define la ruta data/gestion\_tareas.db (relativa a la carpeta desde la que se ejecuta el programa), crea el directorio si no existe y las tablas:
  - tareas(id INTEGER PRIMARY KEY, titulo, departamento, urgencia, tiempo\_estimado, fecha\_entrega, estructura, responsable\_id)
  - empleados(id TEXT PRIMARY KEY, nombre, departamento)

Incluye una migración que agrega la columna responsable\_id si la base de datos fue creada con una versión anterior del sistema (consulta PRAGMA table\_info).

- **TareaRepositorio.guardarTodas(...)** / **EmpleadoRepositorio.guardarTodos(...):** dentro de una transacción, borran el contenido de la tabla e insertan el estado actual con sentencias preparadas por lotes (addBatch / executeBatch); ante un error se hace rollback. De esta forma, la base de datos siempre refleja exactamente las tareas activas y los empleados vigentes.
- **TareaRepositorio.cargarTodas(buscarEmpleado):** reconstruye las tareas con su ID original y resuelve el Responsable Directo mediante la función recibida (búsqueda en el árbol de empleados); si el empleado ya no existe o no pertenece al departamento, la tarea queda "Sin Asignar".
- **EmpleadoRepositorio.cargarTodos():** devuelve los empleados ordenados por ID.

## Complejidad de las principales operaciones

La siguiente tabla resume la complejidad algorítmica de las principales estructuras y operaciones implementadas en el sistema, considerando su funcionamiento principal y los supuestos indicados en cada caso.

| Estructura / Algoritmo                                | Operación                                             | Complejidad                                             |
|-------------------------------------------------------|-------------------------------------------------------|---------------------------------------------------------|
| <b>Pila (Stack)</b>                                   | push / pop / peek                                     | O(1)                                                    |
| <b>Cola (LinkedList)</b>                              | enqueue / dequeue / front                             | O(1)                                                    |
| <b>Lista (ArrayList)</b>                              | insert                                                | O(1) amortizado                                         |
| <b>Lista (ArrayList)</b>                              | delete por ID / find por departamento                 | O(n)                                                    |
| <b>HashMap</b>                                        | Inserción, búsqueda y eliminación por clave           | O(1) promedio                                           |
| <b>PriorityQueue (heap)</b>                           | Inserción / extracción de la tarea de mayor prioridad | O(log n)                                                |
| <b>PriorityQueue (heap)</b>                           | Consulta de la siguiente tarea (peek)                 | O(1)                                                    |
| <b>PriorityQueue (heap)</b>                           | Copia ordenada para la tabla                          | O(n log n)                                              |
| <b>Árbol Binario de Búsqueda (empleados y tareas)</b> | Inserción, búsqueda y eliminación por ID              | O(h): O(log n) si está balanceado; O(n) en el peor caso |
| <b>Árbol Binario de Búsqueda (empleados)</b>          | Listado por departamento (recorrido inorden)          | O(n)                                                    |
| <b>ABB de tareas</b>                                  | Recorridos inorden, preorden y postorden              | O(n)                                                    |
| <b>ABB de tareas</b>                                  | Balanceo por Divide y Vencerás                        | O(n)                                                    |

|                              |                                                |                                                                                                                                          |
|------------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>QuickSort</b>             | Ordenamiento de tareas por urgencia            | $O(n \log n)$ promedio;<br>$O(n^2)$ en el peor caso                                                                                      |
| <b>Búsqueda binaria</b>      | Búsqueda de una tarea por ID (lista ordenada)  | $O(\log n)$                                                                                                                              |
| <b>Búsqueda secuencial</b>   | Búsqueda de una tarea por ID                   | $O(n)$                                                                                                                                   |
| <b>Recursividad</b>          | Cálculo del tiempo total estimado              | $O(n)$                                                                                                                                   |
| <b>Divide y Vencerás</b>     | Distribución de tareas entre k empleados       | $O(n \cdot (k + n))$ en el peor caso, porque la carga de cada empleado se recalcula recursivamente; profundidad de recursión $O(\log n)$ |
| <b>Grafo dirigido (DFS)</b>  | Validación de ciclo al agregar una dependencia | $O(V + E)$                                                                                                                               |
| <b>Grafo dirigido (Kahn)</b> | Orden de ejecución por etapas                  | $O((V + E) \log V)$ , por el uso de cola de prioridad para desempatar por ID                                                             |
| <b>Persistencia (SQLite)</b> | Guardado del estado completo tras cada cambio  | $O(n)$ en una sola transacción                                                                                                           |

Donde  $n$  representa la cantidad de elementos procesados,  $h$  la altura del árbol,  $k$  el número de empleados del departamento,  $V$  el número de vértices (tareas) del grafo y  $E$  el número de aristas (dependencias).

La tabla permite identificar la relación entre la estructura seleccionada y el tipo de operación que realiza el sistema. La sección Búsquedas del programa hace visible esta diferencia: para un mismo ID, la tabla hash requiere una sola comparación, el ABB balanceado y la búsqueda binaria requieren del orden de  $\log_2 n$  comparaciones, y la búsqueda secuencial puede requerir hasta  $n$  comparaciones.

## Flujo de Ejecución

1. GestionTareasApp.main() se ejecuta dentro de SwingUtilities.invokeLater().
2. Se instancian las estructuras base del Modelo (PilaTareas, ColaTareas, ListaTareas).
3. Se instancia GestionTareasView, que construye la cabecera, el menú lateral y las 11 secciones con sus tablas, botones y visualizaciones, y aplica el tema visual.
4. Se instancia GestionTareasController, pasándole la Vista y las tres estructuras base. Dentro de su constructor:
  - se crean las estructuras avanzadas (Cola de Prioridad, Árbol de Empleados, Gestor de Hash/Algoritmos, Grafo de Dependencias y Árbol de Tareas);
  - inicializarPersistencia() abre/crea la base de datos SQLite y carga primero los empleados (árbol, tabla hash y lista) y después las tareas (tabla hash y estructura correspondiente, con su Responsable Directo);
  - initControlador() conecta cada botón y selector con su listener;
  - refrescarResponsablesDisponibles() llena el selector de Responsable Directo;
  - actualizarTablasYMetricas() refresca tablas, métricas, árbol de búsquedas y grafo.
5. Se registra un WindowListener que invocará controlador.guardarEnBD() al cerrar la ventana.
6. Se llama vista.setVisible(true) y la ventana principal queda visible al usuario.
7. A partir de aquí, el flujo es dirigido por eventos: cada clic dispara el listener correspondiente en el Controlador, que ejecuta la lógica del Modelo, actualiza la Vista y guarda los cambios en la base de datos.
8. Al cerrar la ventana se realiza un último guardado y el proceso Java termina. En la siguiente ejecución se recuperan las tareas activas y los empleados desde la base de datos.

## Decisiones de Diseño e Implementación Relevantes

- **Patrón MVC con capa de persistencia:** la Vista no conoce ninguna estructura de datos ni lógica de negocio; solo expone métodos para leer y mostrar datos. Toda la orquestación vive en el Controlador y el acceso a datos en el paquete persistencia.
- **Persistencia inmediata y segura:** cada operación que modifica datos guarda el estado completo dentro de una transacción; si la carga inicial falla, el guardado se desactiva para no borrar información existente. La base de datos solo guarda tareas activas y empleados; las dependencias del grafo, los contadores de tareas resueltas y el historial de tareas atendidas se reinician en cada ejecución.
- **Responsable Directo con regla de departamento:** una tarea solo puede asignarse a un empleado de su mismo departamento. La regla se aplica en tres niveles: el selector solo ofrece empleados válidos, el Controlador lo vuelve a validar y la clase Tarea lanza una excepción si se intenta violar.
- **Distribución que respeta asignaciones previas:** Divide y Vencerás solo reparte tareas "Sin Asignar", equilibra por horas acumuladas dentro de cada departamento y alerta cuando un departamento tiene tareas pero no tiene personal.
- **Búsqueda unificada y comparativa:** en lugar de varios buscadores con alcances distintos, un solo campo ejecuta los cuatro métodos de búsqueda sobre el mismo ID y muestra cuántas comparaciones necesitó cada uno, con lo que la eficiencia de cada estructura se demuestra con datos del propio sistema.
- **ABB de tareas sincronizado y balanceable:** el árbol refleja siempre las tareas activas. Como los IDs son consecutivos, el árbol tiende a desbalancearse; el sistema lo detecta (altura actual vs. altura mínima) y permite balancearlo con Divide y Vencerás.
- **Grafo siempre acíclico:** en lugar de detectar la dependencia circular hasta calcular el orden, el ciclo se impide en el momento de registrar la dependencia (DFS). Además, solo se relacionan tareas activas del mismo departamento, y las tareas que finalizan salen automáticamente del grafo con sus dependencias.

- **Interfaz orientada al usuario final:** los textos de navegación evitan nombrar directamente los algoritmos internos ("Distribución de Tareas" en vez de "Recursión y Divide y Vencerás"), mientras que las visualizaciones de Búsquedas y Grafo muestran la información técnica necesaria para entender el resultado.
- **IDs autogenerados vs. IDs manuales:** las tareas usan un contador estático autoincremental que continúa a partir del mayor ID guardado; los empleados usan un ID de tipo String ingresado manualmente, por lo que su comparación en el BST es lexicográfica.
- **Consolidado de tareas activas:** obtenerTodasLasTareas() reúne Pila + Cola + Lista + Cola de Prioridad y es la fuente común de las operaciones globales (tiempo total, distribución, búsquedas, grafo, pendientes por empleado y consolidado general).
- **Validación de errores consistente:** las entradas numéricas capturan NumberFormatException; las eliminaciones distinguen entre "ID con formato inválido" e "ID válido pero inexistente"; el grafo distingue entre tarea finalizada, inexistente, de otro departamento, duplicada o que formaría un ciclo.
- **Fecha de entrega flexible pero estricta:** se aceptan varios formatos habituales o la selección en calendario, siempre con validación estricta; si se deja vacía, se usa la fecha del día. Los días restantes se calculan al momento de mostrar la tabla, por lo que siempre están actualizados.
- **Trazabilidad:** las estructuras del Modelo imprimen mensajes en la consola estándar de Java y el Controlador registra cada evento en la consola visible de la interfaz.

### Correspondencia entre los requerimientos y la implementación

| Requerimiento de la actividad                                                              | Implementación en el sistema                                                                                                                                 | Dónde se demuestra                                                            |
|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Cola de prioridades por urgencia y fecha de entrega</b>                                 | ColaPrioridadTareas (PriorityQueue con comparador urgencia → fecha)                                                                                          | Sección "Cola Prioridad" (Caso 4)                                             |
| <b>Árboles binarios para organizar empleados por departamento y optimizar asignaciones</b> | ArbolEmpleados (BST por ID con recorrido inorden filtrado por departamento), usado en el selector de responsable y en la distribución; además ArbolTareasABB | Secciones "Empleados", "Registrar Tarea" y "Búsquedas" (Casos 5 a 8, 13 y 14) |
| <b>Rekursividad para estadísticas (tiempo estimado)</b>                                    | calcularTiempoTotalEstimado, calcularCargaRekursiva, recorridos y altura del árbol                                                                           | "Distribución de Tareas" y "Búsquedas" (Casos 9 y 13)                         |
| <b>Divide y vencerás para distribuir tareas en proyectos grandes</b>                       | distribuirRec (reparto por departamento y carga en horas) y construirBalanceado                                                                              | "Distribución de Tareas" y "Búsquedas" (Casos 10 y 14)                        |
| <b>Tablas hash para tareas y empleados</b>                                                 | GestorTablasHashYAlgoritmos (HashMap de tareas y de empleados)                                                                                               | "Búsquedas" (Casos 11 y 12)                                                   |
| <b>Ordenamiento por prioridad y fecha de entrega</b>                                       | QuickSort por urgencia, heap por urgencia/fecha, pendientes por fecha/urgencia, consolidado por urgencia/departamento                                        | Casos 4, 7, 15 y 19                                                           |
| <b>Búsquedas eficientes de tareas específicas</b>                                          | Hash $O(1)$ , ABB $O(\log n)$ , binaria $O(\log n)$ frente a secuencial $O(n)$ , con conteo de comparaciones                                                 | "Búsquedas" (Casos 11 y 12)                                                   |
| <b>Grafos para dependencias entre tareas</b>                                               | GrafoDependencias (lista de adyacencia, DFS anti-ciclos, Kahn por etapas)                                                                                    | "Grafo Dependencias" (Casos 16 a 18)                                          |

## Instrucciones de Uso / Ejecución

### Requisitos previos

- **Java Development Kit (JDK) 21** o superior instalado y configurado en el PATH (el pom.xml compila con maven.compiler.release = 21).
- **Apache Maven** instalado (o un IDE con soporte integrado: IntelliJ IDEA, Eclipse, NetBeans o VS Code con la extensión de Java).
- Conexión a internet la primera vez, para que Maven descargue la dependencia sqlite-jdbc.

### Opción A: Ejecutar desde un IDE (recomendado)

- Abrir el IDE y seleccionar "Importar proyecto Maven existente".
- Apuntar a la carpeta Proyecto\_Final\_Estructura\_De\_Datos/demo (la que contiene el archivo pom.xml).
- Esperar a que el IDE descargue/indexe las dependencias del proyecto.
- Ubicar el archivo src/main/java/com/example/GestionTareasApp.java.
- Ejecutar (Run) esa clase como aplicación Java (contiene el método main).
- Se abrirá la ventana "Sistema Avanzado de Gestión de Tareas Organizacional".

### Opción B: Ejecutar desde la terminal con Maven

```
bash
cd Proyecto_Final_Estructura_De_Datos/demo
mvn compile exec:java -Dexec.mainClass=com.example.GestionTareasApp
```

Este comando compila el proyecto y lo ejecuta incluyendo automáticamente en el classpath el controlador de SQLite. Si se ejecutara solo con `java -cp target/classes ...`, la aplicación no encontraría el controlador y no podría conectarse a la base de datos.

### Ubicación de la base de datos

La base de datos se crea en `data/gestion_tareas.db`, relativa a la carpeta de trabajo desde la que se ejecuta el programa. Si se ejecuta desde otra carpeta, se creará una base de datos nueva y vacía en esa ubicación.

## Navegación básica dentro de la aplicación

- El botón " ||| " de la esquina superior izquierda colapsa/expande el menú lateral.
- El botón "Pantalla Completa" (o F11) ajusta la ventana a la pantalla; "Ventana Flotante" (o Esc) la restaura.
- El menú lateral permite saltar entre 11 secciones: Menú Principal, Registrar Tarea, Pilas (Urgentes), Colas (Programadas), Listas (Generales), Cola Prioridad, Empleados, Distribución de Tareas, Búsquedas, Grafo Dependencias y Ver Todas / Consola.
- Se recomienda registrar primero a los empleados desde "Empleados", para que puedan elegirse como Responsable Directo y participar en la distribución de tareas.
- Toda tarea se crea desde "Registrar Tarea", eligiendo a qué estructura se asigna (Pila, Cola, Lista o Cola de Prioridad); después puede consultarse y procesarse desde la sección correspondiente.
- El modo oscuro se activa desde "Ver Todas / Consola" con el botón "Cambiar a modo oscuro".

## Documento de Prueba de Ejecución

A continuación, se muestran las capturas de pantalla de diferentes casos de prueba, en esta sección se evidencia la correcta ejecución del programa.

**Nota general:** los IDs de las tareas se autogeneran de forma secuencial y se conservan entre ejecuciones gracias a la base de datos. Si ya existen tareas guardadas de sesiones anteriores, los números de ID que aparezcan en pantalla pueden no coincidir con los usados en estos casos; lo importante es verificar que el comportamiento coincida con el resultado esperado descrito. En los casos se usa la notación #A, #B, etc. para referirse al ID que el sistema asigne a cada tarea.

**Preparación recomendada:** registrar primero los empleados del Caso 5 para que estén disponibles como responsable directo en los demás casos.

## Caso de Prueba 1: Registro y procesamiento en la Pila (LIFO)

**Objetivo:** Verificar que las tareas asignadas a la estructura "Pila" se registren correctamente y se procesen bajo el criterio LIFO.

Pasos:

1. Ir a "Registrar Tarea".
2. Completar el formulario con los datos indicados.
3. Clic en "+ Agregar Tarea al Sistema".
4. Ir a "Pilas (Urgentes)" y verificar que la fila de la tarea muestra el indicador "TOPE: Siguiente".
5. Clic en "Consultar Pila (Peek)".
6. Clic en "Procesar Pila (Pop)".
7. Volver al "Menú Principal".

**Resultado esperado:** La tabla de la Pila muestra la tarea recién creada marcada como TOPE. El "Peek" muestra sus datos completos. Tras el "Pop", la tarjeta "PILA · Tareas urgentes" del Menú Principal muestra "Resueltas: 1" y la consola de eventos registra "[POP] Atendida: Corregir error crítico de facturación".

### Registrar Nueva Tarea

Título de Tarea:

Corregir error crítico de facturación

Departamento:

Finanzas

Responsable Directo :

Sin Asignar (se asignará en Distribución)

Solo aparecen empleados del departamento seleccionado. Si no eliges a nadie, la tarea quedará "Sin Asignar" para el módulo de Distribución.

Urgencia (1-Baja a 5-Crítica):

5

Tiempo Estimado (Horas):

3

Fecha de Entrega:

Formato requerido: yyyy-MM-dd (ejemplo: 2025-12-31). Si se deja vacío, se usará la fecha de hoy.

Elegir Fecha

Asignar a Estructura:

Pila (Urgente)

+ Agregar Tarea al Sistema

**Pilas - Tareas Urgentes · Last In, First Out (LIFO)**

| Turno             | ID | Título           | Departame...  | Responsable Directo | Urgencia | Tiempo (hrs) | Fecha de Entrega y Días Restantes |
|-------------------|----|------------------|---------------|---------------------|----------|--------------|-----------------------------------|
| 1                 | 1  | Actualizar BD    | Sistemas      | <i>Sin Asignar</i>  | 1        | 2            | 24/09/2026 (vencida hace 2 días)  |
| 2                 | 2  | Reiniciar ser... | Sistemas      | <i>Sin Asignar</i>  | 3        | 2            | 30/09/2026 (4 días restantes)     |
| 3                 | 3  | Instalar anti... | Sistemas      | Calixto Isaac       | 2        | 2            | 30/09/2026 (4 días restantes)     |
| 4                 | 12 | Actualizar C...  | Ventas        | <i>Sin Asignar</i>  | 5        | 2            | 29/09/2026 (3 días restantes)     |
| 5                 | 18 | Generar nó...    | Recursos H... | <i>Sin Asignar</i>  | 5        | 1            | 29/09/2026 (3 días restantes)     |
| 6                 | 22 | Generar rep...   | Finanzas      | Galilea Valencia    | 4        | 2            | 30/09/2026 (4 días restantes)     |
| 7                 | 32 | Gestionar d...   | Logística     | <i>Sin Asignar</i>  | 5        | 2            | 29/09/2026 (3 días restantes)     |
| ➡ TOPE: Siguiente | 33 | Corregir err...  | Finanzas      | <i>Sin Asignar</i>  | 5        | 3            | 26/09/2026 (vence hoy)            |

Procesar Pila (Pop)

Consultar Pila (Peek)

**PILA · Tareas urgentes**

**Pendientes: 7 | Resueltas: 1**

## Caso de Prueba 2: Registro y procesamiento en la Cola (FIFO)

**Objetivo:** Verificar el comportamiento FIFO de la Cola de tareas programadas.

Pasos:

1. Registrar tareas en el orden indicado.
2. Ir a "Colas (Programadas)" y verificar las filas.
3. Clic en "Consultar Cola (Front)".
4. Clic en "Procesar Cola (Dequeue)".

**Resultado esperado:** La tarea creada aparece marcada como "FRENTE: Siguierte" y el "Front" muestra sus datos (primera en entrar). Tras el "Dequeue", esa tarea desaparece, "Actualizar inventario" pasa a ser el nuevo frente y el contador de resueltas de la Cola aumenta a 1.

| Colas - Tareas Programadas - First In, First Out (FIFO) |    |                  |               |                     |          |              |                                   |
|---------------------------------------------------------|----|------------------|---------------|---------------------|----------|--------------|-----------------------------------|
| Turno                                                   | ID | Título           | Departame...  | Responsable Directo | Urgencia | Tiempo (hrs) | Fecha de Entrega y Días Restantes |
| ➡ FRENTE: Siguierte                                     | 4  | Resolver tic...  | Sistemas      | Sin Asignar         | 2        | 2            | 30/09/2026 (4 días restantes)     |
| 2                                                       | 5  | Configurar ...   | Sistemas      | Sin Asignar         | 1        | 2            | 30/09/2026 (4 días restantes)     |
| 3                                                       | 11 | Seguir pedi...   | Ventas        | Sin Asignar         | 1        | 2            | 30/09/2026 (4 días restantes)     |
| 4                                                       | 15 | Revisar curr...  | Recursos H... | Sin Asignar         | 1        | 2            | 30/09/2026 (4 días restantes)     |
| 5                                                       | 16 | Programar ...    | Recursos H... | Sin Asignar         | 1        | 2            | 30/09/2026 (4 días restantes)     |
| 6                                                       | 20 | Registrar fa...  | Finanzas      | Sin Asignar         | 1        | 2            | 28/09/2026 (2 días restantes)     |
| 7                                                       | 25 | Emitir recib...  | Finanzas      | Sin Asignar         | 1        | 1            | 30/09/2026 (4 días restantes)     |
| 8                                                       | 28 | Actualizar i...  | Logística     | Stephanie Lozano    | 2        | 2            | 30/09/2026 (4 días restantes)     |
| 9                                                       | 29 | Recibir mer...   | Logística     | Sin Asignar         | 1        | 2            | 30/09/2026 (4 días restantes)     |
| 10                                                      | 31 | Verificar sto... | Logística     | Sin Asignar         | 1        | 2            | 30/09/2026 (4 días restantes)     |

Procesar Cola (Dequeue)Consultar Cola (Front)

COLA - Tareas programadas

Pendientes: 9 | Resueltas: 1

### Caso de Prueba 3: Lista General: inserción, búsqueda por departamento y eliminación (válida e inválida)

**Objetivo:** Verificar inserción, filtrado por departamento y eliminación por ID en la Lista General, incluyendo los mensajes de validación.

Pasos:

1. Registrar tareas.
2. Ir a "Listas (Generales)" y confirmar las filas.
3. Seleccionar "Recursos Humanos" y hacer clic en "Buscar por Depto".
4. Clic en "Eliminar por ID" e ingresar algún ID.
5. Repetir "Eliminar por ID" con el mismo ID ya eliminado.
6. Repetir "Eliminar por ID" con un valor no numérico, por ejemplo, abc.

**Resultado esperado:** La búsqueda por departamento lista tareas. Tras el paso 4, la tabla muestra únicamente el resto de tareas registradas. En el paso 5 aparece "No existe ninguna tarea con ID: X en la Lista General." En el paso 6 aparece "Ingrese un ID numérico válido." En ninguno de los dos últimos casos se modifica la tabla ni el contador de resueltas.

| Listas - Tareas Generales |                      |                  |                     |          |              |                                   |
|---------------------------|----------------------|------------------|---------------------|----------|--------------|-----------------------------------|
| ID                        | Título               | Departamento     | Responsable Directo | Urgencia | Tiempo (hrs) | Fecha de Entrega y Días Restantes |
| 6                         | Revisar logs de e... | Sistemas         | Sin Asignar         | 5        | 2            | 30/09/2026 (4 días restantes)     |
| 9                         | Llamar a cliente ... | Ventas           | Sin Asignar         | 2        | 2            | 30/09/2026 (4 días restantes)     |
| 10                        | Enviar cotización    | Ventas           | Sin Asignar         | 1        | 2            | 30/09/2026 (4 días restantes)     |
| 14                        | Revisar metas de...  | Ventas           | Sin Asignar         | 1        | 3            | 30/09/2026 (4 días restantes)     |
| 19                        | Capacitar nuevo...   | Recursos Huma... | Sin Asignar         | 1        | 2            | 25/09/2026 (vencida hace 1 día)   |
| 21                        | Aprobar gasto        | Finanzas         | Sin Asignar         | 1        | 2            | 30/09/2026 (4 días restantes)     |

Departamento: Sistemas Eliminar por ID Buscar por Depto Ordenar Tareas por Urgencia

Búsqueda Depto ×

=== TAREAS EN DEPARTAMENTO Recursos Humanos ===

ID: 19| Capacitar nuevo empleado | Depto: Recursos Humanos | Responsable: Sin Asignar | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 25/09/2026 (vencida hace 1 día) | Tipo: Lista (General)

Aceptar

|    |                    |                  |             |   |   |                                 |
|----|--------------------|------------------|-------------|---|---|---------------------------------|
| 19 | Capacitar nuevo... | Recursos Huma... | Sin Asignar | 1 | 2 | 25/09/2026 (vencida hace 1 día) |
| 21 | Aprobar gasto      | Finanzas         | Sin Asignar | 1 | 2 | 30/09/2026 (4 días restantes)   |

Entrada ×

?

ID a eliminar:

19

Aceptar Cancelar

Listas - Tareas Generales

| ID | Título               | Departamento | Responsable Directo | Urgencia | Tiempo (hrs) | Fecha de Entrega y Días Restantes |
|----|----------------------|--------------|---------------------|----------|--------------|-----------------------------------|
| 6  | Revisar logs de e... | Sistemas     | Sin Asignar         | 5        | 2            | 30/09/2026 (4 días restantes)     |
| 9  | Llamar a cliente ... | Ventas       | Sin Asignar         | 2        | 2            | 30/09/2026 (4 días restantes)     |
| 10 | Enviar cotización    | Ventas       | Sin Asignar         | 1        | 2            | 30/09/2026 (4 días restantes)     |
| 14 | Revisar metas de...  | Ventas       | Sin Asignar         | 1        | 3            | 30/09/2026 (4 días restantes)     |
| 21 | Aprobar gasto        | Finanzas     | Sin Asignar         | 1        | 2            | 30/09/2026 (4 días restantes)     |

Atención ×

No existe ninguna tarea con ID: 19 en la Lista General.

Aceptar

Atención ×

Ingrese un ID numérico válido.

Aceptar

## Caso de Prueba 4: Cola de Prioridad (Heap) y desempate por fecha

**Objetivo:** Verificar que la Cola de Prioridad ordene, extraiga y muestre las tareas por mayor urgencia y, en empate, por fecha de entrega más próxima.

Pasos:

1. Registrar tareas (en cualquier orden).
2. Ir a "Cola Prioridad" y verificar la tabla.
3. Clic en "Consultar Siguiente (Peek)".
4. Clic en "Extraer Mayor Prioridad (Poll)" y verificar la tabla nuevamente.

**Resultado esperado:** La tabla muestra primero "Falla en servidor de producción". El "Peek" confirma esa tarea. Tras el "Poll", la tarea se retira y la tabla solo muestra el resto de tareas.

| Cola de Prioridad - Ordenada por Urgencia y Fecha |                     |                  |                     |          |              |                                   |
|---------------------------------------------------|---------------------|------------------|---------------------|----------|--------------|-----------------------------------|
| ID                                                | Título              | Departamento     | Responsable Directo | Urgencia | Tiempo (hrs) | Fecha de Entrega y Días Restantes |
| 23                                                | Pagar proveedor...  | Finanzas         | Sin Asignar         | 5        | 2            | 29/09/2026 (3 días restantes)     |
| 26                                                | Preparar pedido     | Logística        | Stephanie Lozano    | 3        | 2            | 29/09/2026 (3 días restantes)     |
| 27                                                | Coordinar envío     | Logística        | Joel Balderrama     | 3        | 2            | 30/09/2026 (4 días restantes)     |
| 24                                                | Calcular impues...  | Finanzas         | Sin Asignar         | 3        | 2            | 30/09/2026 (4 días restantes)     |
| 8                                                 | Crear usuario nu... | Sistemas         | Sin Asignar         | 2        | 2            | 29/09/2026 (3 días restantes)     |
| 7                                                 | Restaurar backup    | Sistemas         | Sin Asignar         | 2        | 2            | 30/09/2026 (4 días restantes)     |
| 13                                                | Preparar propue...  | Ventas           | Sin Asignar         | 2        | 3            | 30/09/2026 (4 días restantes)     |
| 17                                                | Procesar vacacio... | Recursos Huma... | Sin Asignar         | 1        | 2            | 29/09/2026 (3 días restantes)     |
| 30                                                | Programar ruta ...  | Logística        | Sin Asignar         | 1        | 2            | 29/09/2026 (3 días restantes)     |

Extraer Mayor Prioridad (Poll)Consultar Siguiente (Peek)

Peek Prioridad

 Siguiente tarea con mayor prioridad:  
[ID: 23] Pagar proveedores | Depto: Finanzas | Responsable: Sin Asignar | Urgencia: 5 | Tiempo Est: 2 hrs | Entrega: 29/09/2026 (3 días restantes) | Tipo: Cola de Prioridad (UrgenciaFecha)

Aceptar

## Caso de Prueba 5: Registro y búsqueda de empleados en el árbol de empleados

**Objetivo:** Verificar la inserción y la búsqueda por ID de empleados.

Pasos:

1. Ir a "Empleados".
2. Registrar los empleados en el orden indicado con "+ Registrar Empleado".
3. Escribir "E001" en el campo ID y hacer clic en "Buscar por ID".
4. Repetir la búsqueda con un ID inexistente, por ejemplo "E999".

**Resultado esperado:** Cada inserción agrega una fila a la tabla de empleados (con 0 tareas pendientes). La búsqueda de "E001" muestra un diálogo con los datos de "Calixto Isaac"; la de "E999" muestra "No se encontró ningún empleado con ID: E999".

The screenshot displays a web application for employee management. At the top, there is a "Registrar Empleado" form with fields for "ID Empleado:", "Nombre Completo:", and "Departamento:" (set to "Sistemas"). A blue button labeled "+ Registrar Empleado" is at the bottom right of the form.

Below the form is a search bar with "ID: E001" and buttons for "Buscar por ID", "Eliminar por ID", "Depto: Sistemas", "Listar por Depto", and "Ver Todos".

The main section is titled "Empleados Registrados" and contains a table with the following data:

| ID   | Nombre Empleado  | Departamento     | Tareas Pendientes |
|------|------------------|------------------|-------------------|
| E001 | Calixto Isaac    | Sistemas         | 1                 |
| E002 | Kevin Ramos      | Sistemas         | 0                 |
| E003 | Antonio Anaya    | Ventas           | 0                 |
| E004 | Uziel Armando    | Ventas           | 0                 |
| E005 | Galilea Valencia | Finanzas         | 1                 |
| E006 | Stephanie Lozano | Logística        | 2                 |
| E007 | Sara Carillo     | Recursos Humanos | 0                 |

Below the table, there is a section "Selecciona un empleado para ver sus tareas pendientes." with a sub-table "Tareas Pendientes del Empleado" showing columns for "ID" and "Título".

Two search dialog boxes are overlaid on the interface:

- The first dialog, titled "Búsqueda BST", shows an information icon and the text: "Empleado Encontrado en Árbol BST: ID: E001, Nombre: Calixto Isaac, Depto: Sistemas". It has an "Aceptar" button.
- The second dialog, also titled "Búsqueda BST", shows a warning icon and the text: "No se encontró ningún empleado con ID: 999". It also has an "Aceptar" button.

## Caso de Prueba 6: Listado de empleados por departamento (orden ascendente por ID)

**Objetivo:** Verificar que el listado filtrado por departamento retorne los empleados ordenados ascendentemente por ID.

**Datos de entrada:** Los mismos empleados del Caso 5.

Pasos:

1. Seleccionar "Sistemas" en el filtro "Depto" y hacer clic en "Listar por Depto".
2. Clic en "Ver Todos".

**Resultado esperado:** En el paso 1 la tabla muestra únicamente "E001 – Calixto Isaac" y "E002 – Kevin Ramos", en ese orden, y la consola registra "Se encontraron 2 empleados en Sistemas". En el paso 2 vuelven a mostrarse todos los empleados.

| ID: <input type="text"/> | Buscar por ID   | Eliminar por ID | Depto: <input type="text" value="Sistemas"/> | Listar por Depto | Ver Todos |
|--------------------------|-----------------|-----------------|----------------------------------------------|------------------|-----------|
| Empleados Registrados    |                 |                 |                                              |                  |           |
| ID                       | Nombre Empleado | Departamento    | Tareas Pendientes                            |                  |           |
| E001                     | Calixto Isaac   | Sistemas        | 1                                            |                  |           |
| E002                     | Kevin Ramos     | Sistemas        | 0                                            |                  |           |

| ID: <input type="text"/> | Buscar por ID    | Eliminar por ID  | Depto: <input type="text" value="Sistemas"/> | Listar por Depto | Ver Todos |
|--------------------------|------------------|------------------|----------------------------------------------|------------------|-----------|
| Empleados Registrados    |                  |                  |                                              |                  |           |
| ID                       | Nombre Empleado  | Departamento     | Tareas Pendientes                            |                  |           |
| E001                     | Calixto Isaac    | Sistemas         | 1                                            |                  |           |
| E002                     | Kevin Ramos      | Sistemas         | 0                                            |                  |           |
| E003                     | Antonio Anaya    | Ventas           | 0                                            |                  |           |
| E004                     | Uziel Armando    | Ventas           | 0                                            |                  |           |
| E005                     | Galilea Valencia | Finanzas         | 1                                            |                  |           |
| E006                     | Stephanie Lozano | Logística        | 2                                            |                  |           |
| E007                     | Sarai Cecilio    | Recursos Humanos | 0                                            |                  |           |
| E008                     | Matias Hernandez | Finanzas         | 0                                            |                  |           |
| E009                     | Joel Balderrama  | Logística        | 1                                            |                  |           |

## Caso de Prueba 7: Responsable Directo y tareas pendientes por empleado

**Objetivo:** Verificar que el selector de responsable directo solo ofrezca empleados del departamento de la tarea y que el detalle de pendientes de cada empleado se actualice.

**Datos de entrada:** Tarea "Migrar servidor de correo", Sistemas, Urgencia 4, Tiempo 5, Estructura "Lista (General)".

Pasos:

1. Ir a "Registrar Tarea" y seleccionar el departamento "Sistemas"; desplegar "responsable directo".
2. Cambiar el departamento a "Ventas" y desplegar de nuevo el selector.
3. Regresar a "Sistemas", elegir "Calixto Isaac (E001)" y registrar la tarea.
4. Ir a "Empleados" y seleccionar a Calixto Isaac en la tabla.

**Resultado esperado:** En el paso 1 solo aparecen "Sin Asignar", Calixto Isaac y Kevin Ramos; en el paso 2 solo "Sin Asignar", Antonio Anaya y Uziel Armando. En las tablas de tareas, la columna "responsable Directo" muestra "Calixto Isaac". En "Empleados", Calixto Isaac tiene tareas pendientes y el panel inferior muestra el detalle de "Migrar servidor de correo".

|                                |                                                                                                                                      |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Título de Tarea:               | Migrar servidor de correo                                                                                                            |
| Departamento:                  | Sistemas                                                                                                                             |
| Responsable Directo :          | Sin Asignar (se asignará en Distribución)<br>Sin Asignar (se asignará en Distribución)<br>Calixto Isaac (E001)<br>Kevin Ramos (E002) |
| Urgencia (1-Baja a 5-Crítica): |                                                                                                                                      |

|                                |                                                                                                                                        |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Título de Tarea:               | Migrar servidor de correo                                                                                                              |
| Departamento:                  | Ventas                                                                                                                                 |
| Responsable Directo :          | Sin Asignar (se asignará en Distribución)<br>Sin Asignar (se asignará en Distribución)<br>Antonio Anaya (E003)<br>Uziel Armando (E004) |
| Urgencia (1-Baja a 5-Crítica): |                                                                                                                                        |

|    |                     |          |               |   |   |                        |
|----|---------------------|----------|---------------|---|---|------------------------|
| 34 | Migrar servidor ... | Sistemas | Calixto Isaac | 4 | 5 | 26/09/2026 (vence hoy) |
|----|---------------------|----------|---------------|---|---|------------------------|

| Empleados Registrados |                  |                  |                   |
|-----------------------|------------------|------------------|-------------------|
| ID                    | Nombre Empleado  | Departamento     | Tareas Pendientes |
| E001                  | Calixto Isaac    | Sistemas         | 2                 |
| E002                  | Kevin Ramos      | Sistemas         | 0                 |
| E003                  | Antonio Anaya    | Ventas           | 0                 |
| E004                  | Uziel Armando    | Ventas           | 0                 |
| E005                  | Galilea Valencia | Finanzas         | 1                 |
| E006                  | Stephanie Lozano | Logística        | 2                 |
| E007                  | Sarai Cecilio    | Recursos Humanos | 0                 |
| E008                  | Matias Hernandez | Finanzas         | 0                 |
| E009                  | Joel Balderrama  | Logística        | 1                 |

Tareas pendientes de Calixto Isaac (Sistemas): 2

▼ Ocultar detalle

| Tareas Pendientes del Empleado |                           |          |                                   |                 |
|--------------------------------|---------------------------|----------|-----------------------------------|-----------------|
| ID                             | Título                    | Urgencia | Fecha de Entrega y Días Restantes | Estructura      |
| 34                             | Migrar servidor de correo | 4        | 26/09/2026 (vence hoy)            | Lista (General) |
| 3                              | Instalar antivirus        | 2        | 30/09/2026 (4 días restantes)     | Pila (Urgente)  |

## Caso de Prueba 8: Eliminación de un empleado del árbol

**Objetivo:** Verificar la eliminación de un empleado del Árbol Binario de Búsqueda y que sus tareas queden "Sin Asignar".

Pasos:

1. Ir a "Empleados", escribir "E002" en el campo ID (o seleccionar su fila) y hacer clic en "Eliminar por ID".
2. Confirmar la eliminación.
3. Revisar la tabla de empleados y la tabla de la Lista General.

**Resultado esperado:** El diálogo de confirmación indica que Laura de Sistemas se eliminara. Tras confirmar, Laura desaparece de la tabla, la búsqueda por ID "E010" ya no la encuentra.

**Registrar Empleado**

ID Empleado:

E010

Nombre Completo:

Laura

Departamento:

Sistemas

+ Registrar Empleado

ID: E010

Buscar por ID

Eliminar por ID

Depto: Sistemas

Listar por Depto

Ver Todos

**Empleados Registrados**

| ID   | Nombre Empleado  | Departamento     | Tareas Pendientes |
|------|------------------|------------------|-------------------|
| E001 | Calixto Isaac    | Sistemas         | 2                 |
| E002 | Kevin Ramos      | Sistemas         | 0                 |
| E003 | Antonio Anaya    | Ventas           | 0                 |
| E004 | Uziel Armando    | Ventas           | 0                 |
| E005 | Galilea Valencia | Finanzas         | 1                 |
| E006 | Stephanie Lozano | Logística        | 2                 |
| E007 | Sarai Cecilio    | Recursos Humanos | 0                 |
| E008 | Matias Hernandez | Finanzas         | 0                 |
| E009 | Joel Balderrama  | Logística        | 1                 |
| E010 | Laura            | Sistemas         | 0                 |

Confirmar eliminación

⌵



¿Eliminar al empleado Laura (ID E010, Sistemas)?

Si

No

Empleado eliminado

⌵



Empleado Laura eliminado.  
0 tarea(s) quedaron "Sin Asignar".

Aceptar

## Caso de Prueba 9: Cálculo del tiempo total estimado (recursividad)

**Objetivo:** Verificar que la suma recursiva de tiempos estimados considere todas las tareas activas de Pila, Cola, Lista y Cola de Prioridad.

**Datos de entrada:** Las tareas activas de los casos anteriores.

Pasos:

1. Ir a "Distribución de Tareas".
2. Clic en "Calcular Tiempo Total Estimado".

**Resultado esperado:** La tarjeta "TOTAL DE TAREAS ANALIZADAS" muestra el número de tareas activas con el desglose por estructura (Pila · Cola · Lista · Prioridad), y la tarjeta "TIEMPO TOTAL ESTIMADO ACUMULADO" muestra la suma exacta de sus horas estimadas.

The screenshot displays a web interface titled "Distribución de Tareas". At the top, there are two buttons: "Calcular Tiempo Total Estimado" and "Distribuir Tareas entre Empleados". Below these, there are two main data cards. The left card, titled "TOTAL DE TAREAS ANALIZADAS", shows "31 tareas activas" and a breakdown: "Pila 7 · Cola 9 · Lista 6 · Prioridad 9". The right card, titled "TIEMPO TOTAL ESTIMADO ACUMULADO", shows "65 horas" and a calculation: "≈ 8.1 jornadas de 8 h · promedio 2.1 h por tarea". At the bottom, there is a section titled "Distribución de Tareas por Empleado" with a note: "Pulsa 'Distribuir Tareas entre Empleados' para ver cómo queda repartido el trabajo."

| Distribución de Tareas                                                                                                            |                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <div>Calcular Tiempo Total Estimado Distribuir Tareas entre Empleados</div>                                                       |                                                                                                               |
| <b>TOTAL DE TAREAS ANALIZADAS</b><br><b>31</b> tareas activas<br>Pila 7 · Cola 9 · Lista 6 · Prioridad 9                          | <b>TIEMPO TOTAL ESTIMADO ACUMULADO</b><br><b>65</b> horas<br>≈ 8.1 jornadas de 8 h · promedio 2.1 h por tarea |
| <b>Distribución de Tareas por Empleado</b><br>Pulsa "Distribuir Tareas entre Empleados" para ver cómo queda repartido el trabajo. |                                                                                                               |

## Caso de Prueba 10: Distribución equilibrada de tareas entre empleados (Divide y Vencerás)

**Objetivo:** Verificar que las tareas sin responsable se repartan entre los empleados de su mismo departamento, equilibrando la carga en horas, y que se respeten los responsables ya asignados.

**Datos de entrada:** Registrar de nuevo a "E002 – Laura Méndez" (Sistemas). Tener al menos tres tareas activas "Sin Asignar" de Sistemas con distintos tiempos, una tarea de Sistemas con responsable directo y al menos una tarea de un departamento sin empleados (por ejemplo, Finanzas o Logística).

Pasos:

1. Ir a "Distribución de Tareas".
2. Clic en "Distribuir Tareas entre Empleados".
3. Revisar las tablas de tareas y la sección "Empleados".

**Resultado esperado:** Se muestra un bloque por empleado con su número de tareas y horas de carga; las tareas de Sistemas se reparten entre Calixto Isaac y Kevin Ramos de manera que sus horas queden equilibradas. La columna "Asignación" indica "Directa" para la tarea que ya tenía responsable y "Nueva" para las asignadas en esta distribución.

**SISTEMA AVANZADO DE GESTIÓN DE TAREAS ORGANIZACIONAL**

Menú Principal  
Registrar Tarea  
Pilas (Urgentes)  
Colas (Programadas)  
Listas (Generales)  
Cola Prioridad  
Empleados  
**Distribución de Tareas**  
Búsquedas  
Grafo Dependencias  
Ver Todas / Consola

**Distribución de Tareas**

Calcular Tiempo Total Estimado Distribuir Tareas entre Empleados

TOTAL DE TAREAS ANALIZADAS  
**37** tareas activas  
Pila 7 - Cola 10 - Lista 8 - Prioridad 12

TIEMPO TOTAL ESTIMADO ACUMULADO  
**80** horas  
= 10.0 jornadas de 8 h - promedio 2.2 h por tarea

**Distribución de Tareas por Empleado**  
11 empleado(s) · 37 tarea(s) asignada(s) · 31 asignada(s) en esta distribución

**Galilea Valencia**  
ID E005 - Finanzas 3 tareas 5 h de carga

| ID | Título                  | Departamento | Urgencia | Tiempo Est. | Fecha de Entrega y Días Restantes | Estructura        | Asignación |
|----|-------------------------|--------------|----------|-------------|-----------------------------------|-------------------|------------|
| 23 | Pagar proveedores       | Finanzas     | 5        | 2 h         | 29/09/2026 (3 días restantes)     | Cola de Priori... | Nueva      |
| 22 | Generar reporte mensual | Finanzas     | 4        | 2 h         | 30/09/2026 (4 días restantes)     | Pila (Urgente)    | Directa    |
| 25 | Emitir recibo de pago   | Finanzas     | 1        | 1 h         | 30/09/2026 (4 días restantes)     | Cola (Secuen...   | Nueva      |

**Matias Hernandez**  
ID E008 - Finanzas 3 tareas 6 h de carga

| ID | Título             | Departamento | Urgencia | Tiempo Est. | Fecha de Entrega y Días Restantes | Estructura        | Asignación |
|----|--------------------|--------------|----------|-------------|-----------------------------------|-------------------|------------|
| 20 | Registrar factura  | Finanzas     | 1        | 2 h         | 28/09/2026 (2 días restantes)     | Cola (Secuen...   | Nueva      |
| 21 | Aprobar gasto      | Finanzas     | 1        | 2 h         | 30/09/2026 (4 días restantes)     | Lista (General)   | Nueva      |
| 24 | Calcular impuestos | Finanzas     | 3        | 2 h         | 30/09/2026 (4 días restantes)     | Cola de Priori... | Nueva      |

**Joel Balderrama**  
ID E009 - Logística 4 tareas 8 h de carga

| ID | Título                    | Departamento | Urgencia | Tiempo Est. | Fecha de Entrega y Días Restantes | Estructura        | Asignación |
|----|---------------------------|--------------|----------|-------------|-----------------------------------|-------------------|------------|
| 32 | Gestionar devolución      | Logística    | 5        | 2 h         | 29/09/2026 (3 días restantes)     | Pila (Urgente)    | Nueva      |
| 30 | Programar ruta de entrega | Logística    | 1        | 2 h         | 29/09/2026 (3 días restantes)     | Cola de Priori... | Nueva      |

## Caso de Prueba 11: Búsqueda de una tarea por ID y comparativa de métodos

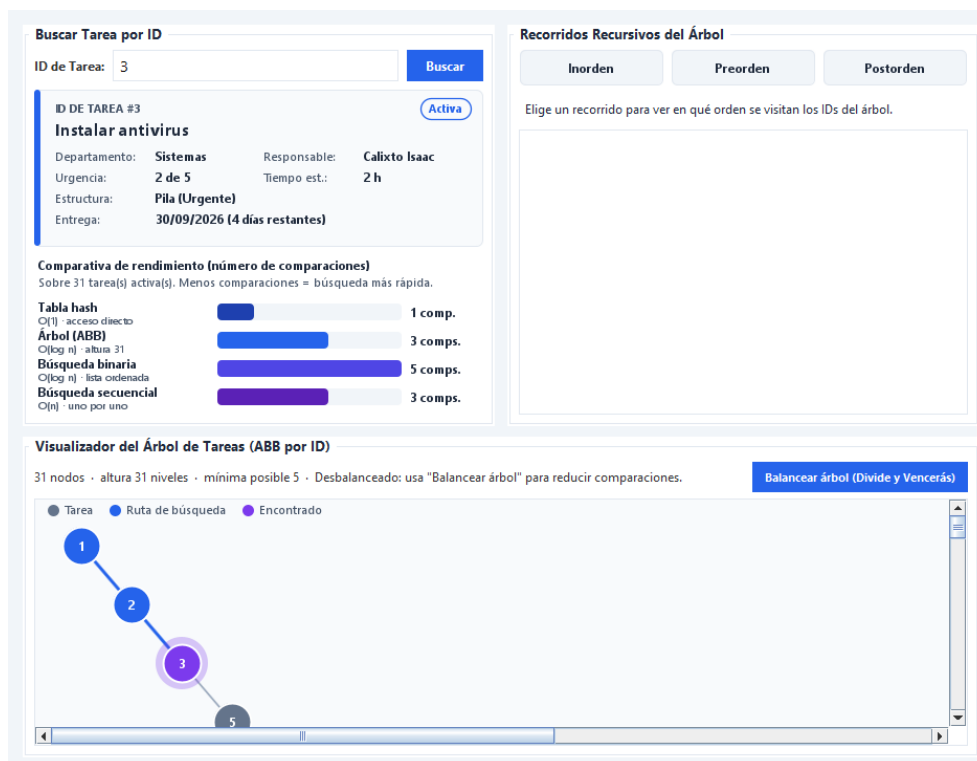
**Objetivo:** Verificar la búsqueda por ID con tabla hash, árbol binario de búsqueda, búsqueda binaria y búsqueda secuencial, y comparar el número de comparaciones de cada método.

**Datos de entrada:** El ID de una tarea activa (preferentemente uno de los más altos) y el ID de una tarea ya procesada en esta sesión (por ejemplo, la extraída en el Caso 4).

Pasos:

1. Ir a "Búsquedas".
2. Escribir el ID de la tarea activa y presionar Enter o "Buscar".
3. Repetir con el ID de la tarea ya procesada.

**Resultado esperado:** En el paso 2, la tarjeta muestra los datos de la tarea con estado "Activa"; la gráfica muestra 1 comparación para la tabla hash, unas pocas para el ABB y la búsqueda binaria, y la mayor cantidad para la búsqueda secuencial; en el dibujo del árbol se resalta la ruta seguida y el nodo encontrado aparece en morado. En el paso 3, la tarea aparece con estado "Ya atendida": solo la tabla hash la encuentra (porque conserva las tareas de la sesión) y los demás métodos indican "no encontrado".



Buscar Tarea por ID

ID de Tarea: 4

Buscar

ID 4 no encontrado

No hay ninguna tarea activa con ese ID.

Comparativa de rendimiento (número de comparaciones)

Sobre 31 tarea(s) activa(s). Menos comparaciones = búsqueda más rápida.

Tabla hash

$O(1)$  · acceso directo

1 comp.

no encontrado

Árbol (ABB)

$O(\log n)$  · altura 31

4 comps.

no encontrado

Búsqueda binaria

$O(\log n)$  · lista ordenada

5 comps.

no encontrado

Búsqueda secuencial

$O(n)$  · uno por uno

31 comps.

no encontrado

Recorridos Recursivos del Árbol

Inorden

Preorden

Postorden

Elige un recorrido para ver en qué orden se visitan los IDs del árbol.

Visualizador del Árbol de Tareas (ABB por ID)

31 nodos · altura 31 niveles · mínima posible 5 · Desbalanceado: usa "Balancear árbol" para reducir comparaciones.

Balancear árbol (Divide y Vencerás)

Tarea

Ruta de búsqueda

Encontrado

1

2

3

5

## Caso de Prueba 12: Búsqueda de un ID inexistente o no válido

**Objetivo:** Verificar el manejo de IDs inexistentes o con formato inválido.

Pasos:

1. En "Búsquedas", buscar el ID 999.
2. Buscar el texto abc.

**Resultado esperado:** Para 999, la tarjeta muestra "ID 999 no encontrado – No hay ninguna tarea activa con ese ID." y todos los métodos muestran "no encontrado" (la búsqueda secuencial revisó todas las tareas). Para abc aparece el aviso "Ingresa un ID de tarea numérico válido."

**Buscar Tarea por ID**

ID de Tarea: 999 Buscar

**ID 999 no encontrado**  
No hay ninguna tarea activa con ese ID.

**Comparativa de rendimiento (número de comparaciones)**  
Sobre 31 tarea(s) activa(s). Menos comparaciones = búsqueda más rápida.

|                                                         |             |                            |
|---------------------------------------------------------|-------------|----------------------------|
| <b>Tabla hash</b><br>$O(1)$ - acceso directo            | <div></div> | 1 comp.<br>no encontrado   |
| <b>Árbol (ABB)</b><br>$O(\log n)$ - altura 31           | <div></div> | 31 comps.<br>no encontrado |
| <b>Búsqueda binaria</b><br>$O(\log n)$ - lista ordenada | <div></div> | 5 comps.<br>no encontrado  |
| <b>Búsqueda secuencial</b><br>$O(n)$ - uno por uno      | <div></div> | 31 comps.<br>no encontrado |

**Recorridos Recursivos del Árbol**

Inorden Preorden Postorden

Elige un recorrido para ver en qué orden se visitan los IDs del árbol.

**Buscar Tarea por ID**

ID de Tarea: abc Buscar

**Atención** ×

Ingresa un ID de tarea numérico válido.

Aceptar

### Caso de Prueba 13: Recorridos recursivos del árbol de tareas

**Objetivo:** Verificar los recorridos inorden, preorden y postorden del ABB de tareas.

Pasos:

1. En "Búsquedas", seleccionar "Inorden".
2. Seleccionar "Preorden".
3. Seleccionar "Postorden".

**Resultado esperado:** Cada opción muestra su explicación y la secuencia de IDs numerada, y en el dibujo del árbol cada nodo muestra su número de visita. Inorden devuelve los IDs de menor a mayor; preorden inicia con la raíz; postorden termina con la raíz.

#### Recorridos Recursivos del Árbol

Inorden (Izquierda → Raíz → Derecha): en un Árbol Binario de Búsqueda devuelve los IDs ordenados de menor a mayor, porque todo lo menor queda a la izquierda y todo lo mayor a la derecha.

Secuencia inorden (31 IDs):

1 → 2 → 3 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 → 13 → 14 → 15 →  
16 → 17 → 18 → 20 → 21 → 22 → 23 → 24 → 25 → 26 → 27 → 28 → 29 → 30 →  
31 → 32 → 34

#### Recorridos Recursivos del Árbol

Preorden (Raíz → Izquierda → Derecha): visita primero cada raíz y después sus subárboles. El primer ID siempre es la raíz del árbol; sirve para copiar o reconstruir el árbol con la misma forma.

Secuencia preorden (31 IDs):

17 → 9 → 5 → 2 → 1 → 3 → 7 → 6 → 8 → 13 → 11 → 10 → 12 → 15 →  
14 → 16 → 26 → 22 → 20 → 18 → 21 → 24 → 23 → 25 → 30 → 28 → 27 → 29 →  
32 → 31 → 34

#### Recorridos Recursivos del Árbol

Postorden (Izquierda → Derecha → Raíz): visita los hijos antes que el padre, así que la raíz queda al final. Es el orden que se usa para eliminar el árbol empezando por las hojas.

Secuencia postorden (31 IDs):

1 → 3 → 2 → 6 → 8 → 7 → 5 → 10 → 12 → 11 → 14 → 16 → 15 → 13 →  
9 → 18 → 21 → 20 → 23 → 25 → 24 → 22 → 27 → 29 → 28 → 31 → 34 → 32 →  
30 → 26 → 17

## Caso de Prueba 14: Balanceo del árbol de tareas (Divide y Vencerás)

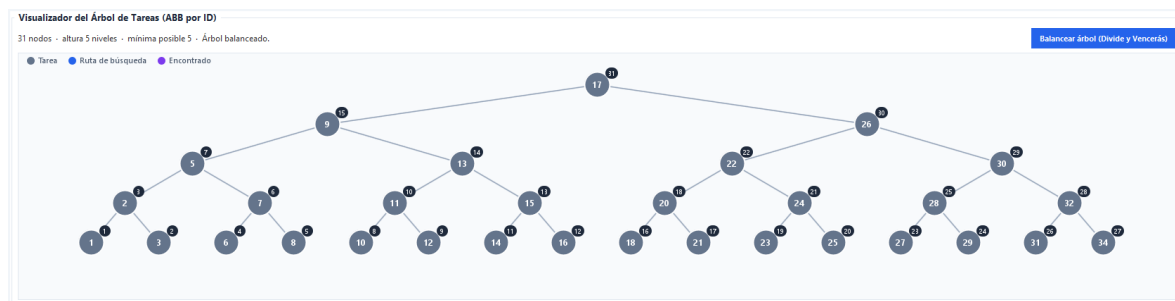
**Objetivo:** Verificar que el balanceo reduzca la altura del árbol a la mínima posible y, con ello, el número de comparaciones.

**Datos de entrada:** Al menos 5 tareas activas.

Pasos:

1. En "Búsquedas", revisar el texto informativo del árbol (nodos, altura y altura mínima posible).
2. Buscar el ID más alto y anotar las comparaciones del ABB.
3. Clic en "Balancear árbol (Divide y Vencerás)".
4. Repetir la búsqueda del paso 2.

**Resultado esperado:** Antes del balanceo el texto indica "Desbalanceado" (altura mayor que la mínima) y el árbol se ve inclinado. Después, la altura coincide con la mínima posible, el texto indica "Árbol balanceado", la consola registra "Árbol balanceado: altura X → Y niveles" y la misma búsqueda requiere menos comparaciones en el ABB.



## Caso de Prueba 15: Ordenamiento por urgencia (QuickSort)

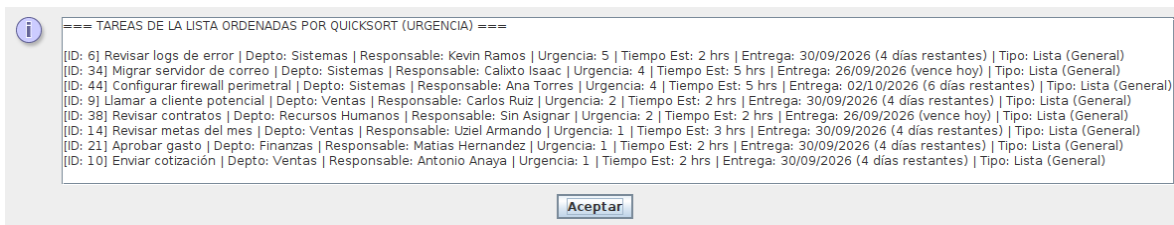
**Objetivo:** Verificar que las tareas de la Lista General se ordenen de mayor a menor urgencia.

**Datos de entrada:** Al menos dos tareas con urgencias distintas en la Lista General.

Pasos:

1. Ir a "Listas (Generales)".
2. Clic en "Ordenar Tareas por Urgencia".

**Resultado esperado:** Se despliega el diálogo "QuickSort Completado" con las tareas ordenadas de mayor a menor urgencia. Si la lista tiene menos de dos tareas, se muestra el aviso correspondiente.



## Caso de Prueba 16: Grafo de Dependencias: orden de ejecución por etapas

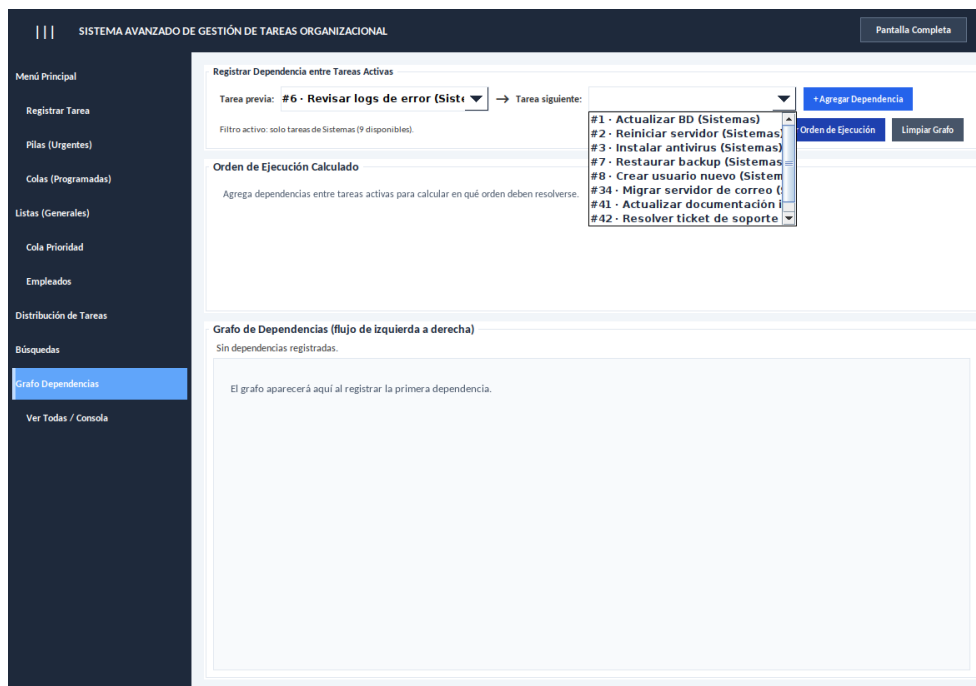
**Objetivo:** Verificar que el sistema calcule una secuencia válida de ejecución y agrupe las tareas por etapas.

**Datos de entrada:** Registrar cuatro tareas de Sistemas: #A "Diseñar base de datos", #B "Desarrollar API", #C "Diseñar interfaz", #D "Pruebas de integración". Dependencias: #A → #B, #A → #C, #B → #D, #C → #D.

Pasos:

1. Ir a "Grafo Dependencias".
2. Elegir la tarea previa y verificar que la lista de la tarea siguiente solo muestra tareas de Sistemas.
3. Registrar las cuatro dependencias con "+ Agregar Dependencia".
4. Clic en "Calcular Orden de Ejecución".

**Resultado esperado:** El panel "Orden de Ejecución Calculado" muestra: Etapa 1: #A; Etapa 2: #B y #C (pueden realizarse en paralelo); Etapa 3: #D. El dibujo del grafo muestra tres columnas con flechas de izquierda a derecha y el número de paso en cada nodo.



SISTEMA AVANZADO DE GESTIÓN DE TAREAS ORGANIZACIONAL

Pantalla Completa

Menú Principal

Registrar Tarea

Pilas (Urgentes)

Colas (Programadas)

Listas (Generales)

Cola Prioridad

Empleados

Distribución de Tareas

Búsquedas

Grafo Dependencias

Ver Todas / Consola

Registrar Dependencia entre Tareas Activas

Tarea previa:

→

Tarea siguiente:

+ Agregar Dependencia

Elige la tarea previa: la lista de la derecha mostrará solo tareas activas de su mismo departamento.

Calcular Orden de Ejecución

Limpiar Grafo

Orden de Ejecución Calculado

Resuelve las tareas en este orden exacto para no bloquear ninguna dependencia · 4 tareas en 3 etapas (las de la misma etapa no dependen entre sí).

1 #6 Etapa 1  
Revisar logs de error

→

2 #2 Etapa 2  
Reiniciar servidor

→

3 #7 Etapa 2  
Restaurar backup

→

4 #1 Etapa 3  
Actualizar BD

Grafo de Dependencias (flujo de izquierda a derecha)

4 tareas conectadas · 4 dependencias · cada flecha va de la tarea previa a la que depende de ella.

ETAPA 1 - puede empezar ya

ETAPA 2

ETAPA 3

#6  
Revisar logs de error  
Paso 1

→

#2  
Reiniciar servidor  
Paso 2

→

#7  
Restaurar backup  
Paso 3

→

#1  
Actualizar BD  
Paso 4

64

### Caso de Prueba 17: Grafo de Dependencias: prevención de dependencia circular

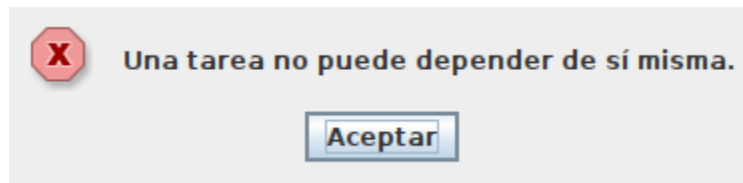
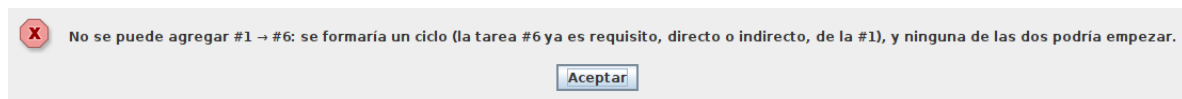
**Objetivo:** Verificar que el sistema impida registrar una dependencia que forme un ciclo.

**Datos de entrada:** Las dependencias del Caso 16 y la nueva dependencia  $\#D \rightarrow \#A$ .

Pasos:

1. Intentar registrar la dependencia  $\#D \rightarrow \#A$ .
2. Intentar registrar  $\#A \rightarrow \#A$ .
3. Intentar registrar de nuevo  $\#A \rightarrow \#B$ .

**Resultado esperado:** En el paso 1 aparece el diálogo "Dependencia no permitida" con el mensaje "No se puede agregar  $\#D \rightarrow \#A$ : se formaría un ciclo...", y el grafo no cambia. En el paso 2 aparece "Una tarea no puede depender de sí misma." y en el paso 3 "La dependencia  $\#A \rightarrow \#B$  ya existe."



## Caso de Prueba 18: Grafo de Dependencias: departamentos distintos y tareas finalizadas

**Objetivo:** Verificar las reglas de negocio del grafo.

Pasos:

1. Escribir manualmente en "tarea previa" el ID de una tarea de Sistemas y en "tarea siguiente" el ID de una tarea de Ventas; clic en "+ Agregar Dependencia".
2. Escribir el ID de una tarea ya procesada.
3. Procesar (Pop, Dequeue, Poll o eliminar) una tarea que forme parte del grafo y regresar a "Grafo Dependencias".

**Resultado esperado:** En el paso 1 aparece "Transición no permitida: Las dependencias solo se pueden establecer entre tareas del mismo departamento". En el paso 2 aparece "La tarea #X ya fue finalizada (salió de su estructura). Solo se pueden usar tareas activas.". En el paso 3 la tarea desaparece del grafo junto con sus dependencias y la consola lo registra.



⚠ Transición no permitida:

Las dependencias solo se pueden establecer entre tareas del mismo departamento (Ejemplo: Tarea #6 [Sistemas] vs Tarea #12 [Ventas]).

Aceptar



La tarea #35 ya fue finalizada (salió de su estructura). Solo se pueden usar tareas activas.

Aceptar

SISTEMA AVANZADO DE GESTIÓN DE TAREAS ORGANIZACIONAL

Pantalla Completa

Menú Principal

Registrar Tarea

Pilas (Urgentes)

Colas (Programadas)

Listas (Generales)

Cola Prioridad

Empleados

Distribución de Tareas

Búsquedas

Grafo de Dependencias

Ver Todas / Consola

Registrar Dependencia entre Tareas Activas

Tarea previa:  → Tarea siguiente: 

+ Agregar Dependencia

Elige la tarea previa: la lista de la derecha mostrará solo tareas activas de su mismo departamento. 

Calcular Orden de Ejecución

Limpiar Grafo

Orden de Ejecución Calculado

Resuelve las tareas en este orden exacto para no bloquear ninguna dependencia · 3 tareas en 2 etapas (las de la misma etapa no dependen entre sí).

1 #2 Etapa 1  
Reiniciar servidor

 → 

2 #7 Etapa 1  
Restaurar backup

 → 

3 #1 Etapa 2  
Actualizar BD

Grafo de Dependencias (flujo de izquierda a derecha)

3 tareas conectadas · 2 dependencias · cada flecha va de la tarea previa a la que depende de ella.

ETAPA 1 - puede empezar ya

ETAPA 2

#2 Paso 1  
Reiniciar servidor

#7 Paso 2  
Restaurar backup

#1 Paso 3  
Actualizar BD

67

## Caso de Prueba 19: Consolidado general, consola y modo oscuro

**Objetivo:** Verificar el listado consolidado de todas las tareas activas ordenadas por urgencia y departamento, la consola de eventos y el modo oscuro.

Pasos:

1. Ir a "Ver Todas / Consola".
2. Clic en "Ver Todas (Ordenadas por Urgencia/Depto)".
3. Clic en "Cambiar a modo oscuro" y navegar por algunas secciones.

**Resultado esperado:** La tabla del consolidado incluye las tareas de Pila, Cola, Lista y Cola de Prioridad con su estructura y responsable. El diálogo muestra las tareas de mayor a menor urgencia y, en empate, en orden alfabético por departamento. La consola muestra el historial de eventos de la sesión. Toda la interfaz cambia a la paleta oscura.

SISTEMA AVANZADO DE GESTIÓN DE TAREAS ORGANIZACIONAL Pantalla Completa

Menú Principal

Registrar Tarea

Pilas (Urgentes)

Colas (Programadas)

Listas (Generales)

Cola Prioridad

Empleados

Distribución de Tareas

Búsquedas

Grafo Dependencias

Ver Todas / Consola

Ver Todas (Ordenadas por Urgencia/Depto) Cambiar a modo oscuro

**Consolidado General de Tareas**

| ID | Título                | Departamento     | Responsable Directo | Urgencia | Fecha de Entrega y Días Restantes | Estructura        |
|----|-----------------------|------------------|---------------------|----------|-----------------------------------|-------------------|
| 1  | Actualizar BD         | Sistemas         | Kevin Ramos         | 1        | 24/09/2026 (vence hace 2 días)    | Pila (Urgente)    |
| 2  | Reiniciar servidor    | Sistemas         | Ana Torres          | 3        | 30/09/2026 (4 días restantes)     | Pila (Urgente)    |
| 3  | Instalar antivirus    | Sistemas         | Calixto Isaac       | 2        | 30/09/2026 (4 días restantes)     | Pila (Urgente)    |
| 12 | Actualizar CRM        | Ventas           | Antonio Anaya       | 5        | 29/09/2026 (3 días restantes)     | Pila (Urgente)    |
| 18 | Generar nómina        | Recursos Humanos | Sin Asignar         | 5        | 29/09/2026 (3 días restantes)     | Pila (Urgente)    |
| 22 | Generar reporte ...   | Finanzas         | Galilea Valencia    | 4        | 30/09/2026 (4 días restantes)     | Pila (Urgente)    |
| 32 | Gestionar devoluc...  | Logística        | Joel Balderrama     | 5        | 29/09/2026 (3 días restantes)     | Pila (Urgente)    |
| 11 | Seguir pedido pen...  | Ventas           | Uziel Armando       | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 15 | Revisar currículums   | Recursos Humanos | Sin Asignar         | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 16 | Programar entrev...   | Recursos Humanos | Sin Asignar         | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 20 | Registrar factura     | Finanzas         | Matias Hernandez    | 1        | 28/09/2026 (2 días restantes)     | Cola (Secuencial) |
| 25 | Emitir recibo de p... | Finanzas         | Galilea Valencia    | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 28 | Actualizar inventa... | Logística        | Stephanie Lozano    | 2        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 29 | Recibir mercancía     | Logística        | Stephanie Lozano    | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 31 | Verificar stock       | Logística        | Joel Balderrama     | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 36 | Enviar reporte me...  | Ventas           | Carlos Ruiz         | 2        | 26/09/2026 (vence hoy)            | Cola (Secuencial) |
| 37 | Actualizar inventa... | Logística        | Stephanie Lozano    | 3        | 26/09/2026 (vence hoy)            | Cola (Secuencial) |
| 9  | Llamar a cliente p... | Ventas           | Carlos Ruiz         | 2        | 30/09/2026 (4 días restantes)     | Lista (General)   |
| 40 | Enviar notificación   | Ventas           | Antonio Anaya       | 4        | 30/09/2026 (4 días restantes)     | Lista (General)   |

**Consola de Eventos en Tiempo Real**

```
Responsable Directo: Sin Asignar | Entrega: 15/10/2026 (19 días restantes)
[RECORRIDO POSTORDEN] [1, 3, 2, 10, 9, 8, 7, 12, 15, 14, 13, 17, 21, 20, 18, 16, 11, 23, 25, 24, 27, 30, 29, 28, 26, 32, 37, 36]
[INSERT] Tarea en Lista General: Renovar licencias de software
Responsable Directo: Sin Asignar | Entrega: 26/09/2026 (vence hoy)
[RECORRIDO POSTORDEN] [1, 3, 2, 10, 9, 8, 7, 12, 15, 14, 13, 17, 21, 20, 18, 16, 11, 23, 25, 24, 27, 30, 29, 28, 26, 32, 37, 36]
```

=== CONSOLIDADO GENERAL DE TAREAS ===

[ID: 23] Pagar proveedores | Depto: Finanzas | Responsable: Galilea Valencia | Urgencia: 5 | Tiempo Est: 2 hrs | Entrega: 29/09/2026 (3 días restantes) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 32] Gestionar devolución | Depto: Logística | Responsable: Joel Balderrama | Urgencia: 5 | Tiempo Est: 2 hrs | Entrega: 29/09/2026 (3 días restantes) | Tipo: Pila (Urgente)  
[ID: 18] Generar nómina | Depto: Recursos Humanos | Responsable: Sin Asignar | Urgencia: 5 | Tiempo Est: 1 hrs | Entrega: 29/09/2026 (3 días restantes) | Tipo: Pila (Urgente)  
[ID: 12] Actualizar CRM | Depto: Ventas | Responsable: Antonio Anaya | Urgencia: 5 | Tiempo Est: 2 hrs | Entrega: 29/09/2026 (3 días restantes) | Tipo: Pila (Urgente)  
[ID: 22] Generar reporte mensual | Depto: Finanzas | Responsable: Galilea Valencia | Urgencia: 4 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Pila (Urgente)  
[ID: 34] Migrar servidor de correo | Depto: Sistemas | Responsable: Calixto Isaac | Urgencia: 4 | Tiempo Est: 5 hrs | Entrega: 26/09/2026 (vence hoy) | Tipo: Lista (General)  
[ID: 44] Configurar firewall perimetral | Depto: Sistemas | Responsable: Ana Torres | Urgencia: 4 | Tiempo Est: 5 hrs | Entrega: 02/10/2026 (6 días restantes) | Tipo: Lista (General)  
[ID: 45] Renovar contrato con proveedor | Depto: Finanzas | Responsable: Sin Asignar | Urgencia: 3 | Tiempo Est: 2 hrs | Entrega: 31/12/2026 (96 días restantes) | Tipo: Lista (General)  
[ID: 46] Renovar contrato de arrendamiento | Depto: Finanzas | Responsable: Sin Asignar | Urgencia: 3 | Tiempo Est: 2 hrs | Entrega: 31/12/2026 (96 días restantes) | Tipo: Lista (General)  
[ID: 47] Renovar póliza de seguro | Depto: Finanzas | Responsable: Sin Asignar | Urgencia: 3 | Tiempo Est: 2 hrs | Entrega: 15/10/2026 (19 días restantes) | Tipo: Lista (General)  
[ID: 48] Renovar licencias de software | Depto: Finanzas | Responsable: Sin Asignar | Urgencia: 3 | Tiempo Est: 2 hrs | Entrega: 26/09/2026 (vence hoy) | Tipo: Lista (General)  
[ID: 24] Calcular impuestos | Depto: Finanzas | Responsable: Matias Hernandez | Urgencia: 3 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 37] Actualizar inventario de papelería | Depto: Logística | Responsable: Stephanie Lozano | Urgencia: 3 | Tiempo Est: 2 hrs | Entrega: 26/09/2026 (vence hoy) | Tipo: Cola (Secuencial)  
[ID: 26] Preparar pedido | Depto: Logística | Responsable: Stephanie Lozano | Urgencia: 3 | Tiempo Est: 2 hrs | Entrega: 29/09/2026 (3 días restantes) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 27] Coordinar envío | Depto: Logística | Responsable: Joel Balderrama | Urgencia: 3 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 2] Reiniciar servidor | Depto: Sistemas | Responsable: Ana Torres | Urgencia: 3 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Pila (Urgente)  
[ID: 42] Resolver ticket de soporte | Depto: Sistemas | Responsable: Kevin Ramos | Urgencia: 3 | Tiempo Est: 2 hrs | Entrega: 06/10/2026 (10 días restantes) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 43] Negociar contrato con cliente | Depto: Ventas | Responsable: Carlos Ruiz | Urgencia: 3 | Tiempo Est: 3 hrs | Entrega: 28/09/2026 (2 días restantes) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 28] Actualizar inventario | Depto: Logística | Responsable: Stephanie Lozano | Urgencia: 2 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Cola (Secuencial)  
[ID: 38] Revisar contratos | Depto: Recursos Humanos | Responsable: Sin Asignar | Urgencia: 2 | Tiempo Est: 2 hrs | Entrega: 26/09/2026 (vence hoy) | Tipo: Lista (General)  
[ID: 3] Instalar antivirus | Depto: Sistemas | Responsable: Calixto Isaac | Urgencia: 2 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Pila (Urgente)  
[ID: 8] Crear usuario nuevo | Depto: Sistemas | Responsable: Kevin Ramos | Urgencia: 2 | Tiempo Est: 2 hrs | Entrega: 29/09/2026 (3 días restantes) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 7] Restaurar backup | Depto: Sistemas | Responsable: Calixto Isaac | Urgencia: 2 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 36] Enviar reporte mensual | Depto: Ventas | Responsable: Carlos Ruiz | Urgencia: 2 | Tiempo Est: 1 hrs | Entrega: 26/09/2026 (vence hoy) | Tipo: Cola (Secuencial)  
[ID: 9] Llamar a cliente potencial | Depto: Ventas | Responsable: Carlos Ruiz | Urgencia: 2 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Lista (General)  
[ID: 13] Preparar propuesta comercial | Depto: Ventas | Responsable: Antonio Anaya | Urgencia: 2 | Tiempo Est: 3 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 20] Registrar factura | Depto: Finanzas | Responsable: Matias Hernandez | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 28/09/2026 (2 días restantes) | Tipo: Cola (Secuencial)  
[ID: 25] Emitir recibo de pago | Depto: Finanzas | Responsable: Galilea Valencia | Urgencia: 1 | Tiempo Est: 1 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Cola (Secuencial)  
[ID: 21] Aprobar gasto | Depto: Finanzas | Responsable: Matias Hernandez | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Lista (General)  
[ID: 29] Recibir mercancía | Depto: Logística | Responsable: Stephanie Lozano | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Cola (Secuencial)  
[ID: 31] Verificar stock | Depto: Logística | Responsable: Joel Balderrama | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Cola (Secuencial)  
[ID: 30] Programar ruta de entrega | Depto: Logística | Responsable: Joel Balderrama | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 29/09/2026 (3 días restantes) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 15] Revisar currículums | Depto: Recursos Humanos | Responsable: Sin Asignar | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Cola (Secuencial)  
[ID: 16] Programar entrevista | Depto: Recursos Humanos | Responsable: Sin Asignar | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Cola (Secuencial)  
[ID: 17] Procesar vacaciones | Depto: Recursos Humanos | Responsable: Sin Asignar | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 29/09/2026 (3 días restantes) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 1] Actualizar BD | Depto: Sistemas | Responsable: Kevin Ramos | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 24/09/2026 (vencida hace 2 días) | Tipo: Pila (Urgente)  
[ID: 41] Actualizar documentación interna | Depto: Sistemas | Responsable: Ana Torres | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 26/09/2026 (vence hoy) | Tipo: Cola de Prioridad (Urgencia/Fecha)  
[ID: 11] Seguir pedido pendiente | Depto: Ventas | Responsable: Uziel Armando | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Cola (Secuencial)  
[ID: 10] Enviar cotización | Depto: Ventas | Responsable: Antonio Anaya | Urgencia: 1 | Tiempo Est: 2 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Lista (General)  
[ID: 14] Revisar metas del mes | Depto: Ventas | Responsable: Uziel Armando | Urgencia: 1 | Tiempo Est: 3 hrs | Entrega: 30/09/2026 (4 días restantes) | Tipo: Lista (General)

Aceptar

||| SISTEMA AVANZADO DE GESTIÓN DE TAREAS ORGANIZACIONAL

Pantalla Completa

Menú Principal

Ver Todas (Ordenadas por Urgencia/Depto)

Cambiar a modo claro

Registrar Tarea

Pilas (Urgentes)

Colas (Programadas)

Listas (Generales)

Cola Prioridad

Empleados

Distribución de Tareas

Búsquedas

Grafo Dependencias

Ver Todas / Consola

Consolidado General de Tareas

| ID | Título                | Departamento     | Responsable Directo | Urgencia | Fecha de Entrega y Días Restantes | Estructura        |
|----|-----------------------|------------------|---------------------|----------|-----------------------------------|-------------------|
| 1  | Actualizar BD         | Sistemas         | Kevin Ramos         | 1        | 24/09/2026 (vencida hace 2 días)  | Pila (Urgente)    |
| 2  | Reiniciar servidor    | Sistemas         | Ana Torres          | 3        | 30/09/2026 (4 días restantes)     | Pila (Urgente)    |
| 3  | Instalar antivirus    | Sistemas         | Calixto Isaac       | 2        | 30/09/2026 (4 días restantes)     | Pila (Urgente)    |
| 12 | Actualizar CRM        | Ventas           | Antonio Anaya       | 5        | 29/09/2026 (3 días restantes)     | Pila (Urgente)    |
| 18 | Generar nómina        | Recursos Humanos | Sin Asignar         | 5        | 29/09/2026 (3 días restantes)     | Pila (Urgente)    |
| 22 | Generar reporte ...   | Finanzas         | Galilea Valencia    | 4        | 30/09/2026 (4 días restantes)     | Pila (Urgente)    |
| 32 | Gestionar devoluc...  | Logística        | Joel Balderrama     | 5        | 29/09/2026 (3 días restantes)     | Pila (Urgente)    |
| 11 | Seguir pedido pen...  | Ventas           | Uziel Armando       | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 15 | Revisar currículums   | Recursos Humanos | Sin Asignar         | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 16 | Programar entrev...   | Recursos Humanos | Sin Asignar         | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 20 | Registrar factura     | Finanzas         | Matias Hernandez    | 1        | 28/09/2026 (2 días restantes)     | Cola (Secuencial) |
| 25 | Emitir recibo de p... | Finanzas         | Galilea Valencia    | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 28 | Actualizar inventa... | Logística        | Stephanie Lozano    | 2        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 29 | Recibir mercancía     | Logística        | Stephanie Lozano    | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 31 | Verificar stock       | Logística        | Joel Balderrama     | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 36 | Enviar reporte me...  | Ventas           | Carlos Ruiz         | 2        | 26/09/2026 (vence hoy)            | Cola (Secuencial) |
| 37 | Actualizar inventa... | Logística        | Stephanie Lozano    | 3        | 26/09/2026 (vence hoy)            | Cola (Secuencial) |
| 9  | Llamar a cliente p... | Ventas           | Carlos Ruiz         | 2        | 30/09/2026 (4 días restantes)     | Lista (General)   |
| 40 | Enviar cotización     | Ventas           | Antonio Anaya       | 4        | 29/09/2026 (3 días restantes)     | Lista (General)   |

Consola de Eventos en Tiempo Real

Responsable Directo: Sin Asignar | Entrega: 15/10/2026 (19 días restantes)  
[RECORRIDO POSTORDEN] [1, 3, 2, 10, 9, 8, 7, 12, 15, 14, 13, 17, 21, 20, 18, 16, 11, 23, 25, 24, 27, 30, 29, 28, 26, 32, 37, 36]  
[INSERT] Tarea en Lista General: Renovar licencias de software  
Responsable Directo: Sin Asignar | Entrega: 26/09/2026 (vence hoy)  
[RECORRIDO POSTORDEN] [1, 3, 2, 10, 9, 8, 7, 12, 15, 14, 13, 17, 21, 20, 18, 16, 11, 23, 25, 24, 27, 30, 29, 28, 26, 32, 37, 36]

## **Caso de Prueba 20: Captura de fecha de entrega (texto validado y calendario)**

**Objetivo:** Verificar que se aceptan los formatos de fecha admitidos, se rechazan los formatos inválidos y el selector de calendario completa correctamente el campo.

**Datos de entrada:** Tarea "Renovar contrato con proveedor", Finanzas, Urgencia 3.

Pasos:

1. Ir a "Registrar Tarea".
2. Escribir 2026/12/31 en "Fecha de Entrega" y registrar la tarea.
3. Registrar una tarea similar escribiendo 31/12/2026.
4. Registrar una tercera tarea usando "Elegir Fecha", navegando con "<" / ">" y seleccionando un día.
5. Intentar registrar una tarea con un formato inválido (por ejemplo, 31\_12\_2026 o 2026.12.31) o con una fecha inexistente (2026-02-30).
6. Registrar una tarea dejando el campo de fecha vacío.

**Resultado esperado:** En los pasos 2 y 3 la tarea se registra con la fecha 31/12/2026 y la tabla muestra los días restantes. En el paso 4 el campo se completa automáticamente en formato yyyy-MM-dd y la tarea se registra sin errores. En el paso 5 aparece el aviso "La fecha de entrega debe tener un formato válido: yyyy-MM-dd (ejemplo: 2025-12-31), o formatos comunes como 31/12/2025." y la tarea no se registra. En el paso 6 la tarea se registra con la fecha de hoy ("vence hoy").

|||

SISTEMA AVANZADO DE GESTIÓN DE TAREAS ORGANIZACIONAL

Pantalla Completa

Menú Principal

Registrar Tarea

Pilas (Urgentes)

Colas (Programadas)

Listas (Generales)

Cola Prioridad

Empleados

Distribución de Tareas

Búsquedas

Grafo Dependencias

Ver Todas / Consola

Registrar Nueva Tarea

Título de Tarea:

Renovar contrato con proveedor

Departamento:

Finanzas

Responsable Directo :

Sin Asignar (se asignará en Distribución)

Solo aparecen empleados del departamento seleccionado. Si no eliges a nadie, la tarea quedará "Sin Asignar" para el módulo de Distribución.

Urgencia (1-Baja a 5-Crítica):

3

Tiempo Estimado (Horas):

2

Fecha de Entrega:

Formato requerido: yyyy-MM-dd (ejemplo: 2025-12-31). Si se deja vacío, se usará la fecha de hoy.

2026/12/31

Elegir Fecha

Asignar a Estructura:

Lista (General)

+ Agregar Tarea al Sistema

|||

SISTEMA AVANZADO DE GESTIÓN DE TAREAS ORGANIZACIONAL

Pantalla Completa

Menú Principal

Registrar Tarea

Pilas (Urgentes)

Colas (Programadas)

Listas (Generales)

Cola Prioridad

Empleados

Distribución de Tareas

Búsquedas

Grafo Dependencias

Ver Todas / Consola

Registrar Nueva Tarea

Título de Tarea:

Renovar póliza de seguro

Departamento:

Finanzas

Responsable Directo :

Sin Asignar (se asignará en Distribución)

Solo aparecen empleados del departamento seleccionado. Si no eliges a nadie, la tarea quedará "Sin Asignar" para el módulo de Distribución.

Urgencia (1-Baja a 5-Crítica):

3

Tiempo Estimado (Horas):

2

Fecha de Entrega:

Formato requerido: yyyy-MM-dd (ejemplo: 2025-12-31). Si se deja vacío, se usará la fecha de hoy.

Elegir Fecha

Asignar a Estructura:

Lista (General)

+ Agregar Tarea al Sistema

<

Octubre 2026

>

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| Lu | Ma | Mi | Ju | Vi | Sa | Do |
|    |    |    | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  | 10 | 11 |
| 12 | 13 | 14 | 15 | 16 | 17 | 18 |
| 19 | 20 | 21 | 22 | 23 | 24 | 25 |
| 26 | 27 | 28 | 29 | 30 | 31 |    |

|||

SISTEMA AVANZADO DE GESTIÓN DE TAREAS ORGANIZACIONAL

Pantalla Completa

Menú Principal

Registrar Tarea

Pilas (Urgentes)

Colas (Programadas)

Listas (Generales)

Cola Prioridad

Empleados

Distribución de Tareas

Búsquedas

Grafo Dependencias

Ver Todas / Consola

Registrar Nueva Tarea

Título de Tarea:

Renovar póliza de seguro

Departamento:

Finanzas

Responsable Directo:

Sin Asignar (se asignará en Distribución)

Solo aparecen empleados del departamento seleccionado. Si no eliges a nadie, la tarea quedará "Sin Asignar" para el módulo de Distribución.

Urgencia (1-Baja a 5-Critica):

3

Tiempo Estimado (Horas):

2

Fecha de Entrega:

Formato requerido: yyyy-MM-dd (ejemplo: 2025-12-31). Si se deja vacío, se usará la fecha de hoy.

2026-10-15

Elegir Fecha

Asignar a Estructura:

Lista (General)

+ Agregar Tarea al Sistema



La fecha de entrega debe tener un formato válido: yyyy-MM-dd (ejemplo: 2025-12-31), o formatos comunes como 31/12/2025.

Aceptar



La fecha de entrega debe tener un formato válido: yyyy-MM-dd (ejemplo: 2025-12-31), o formatos comunes como 31/12/2025.

Aceptar

SISTEMA AVANZADO DE GESTIÓN DE TAREAS ORGANIZACIONAL

Pantalla Completa

Menú Principal

Registrar Tarea

Pilas (Urgentes)

Colas (Programadas)

Listas (Generales)

Cola Prioridad

Empleados

Distribución de Tareas

Búsquedas

Grafo Dependencias

Ver Todas / Consola

Listas - Tareas Generales

| ID | Título                | Departamento     | Responsable Directo | Urgencia | Tiempo (hrs) | Fecha de Entrega y Días Restantes |
|----|-----------------------|------------------|---------------------|----------|--------------|-----------------------------------|
| 9  | Llamar a cliente p... | Ventas           | Carlos Ruiz         | 2        | 2            | 30/09/2026 (4 días restantes)     |
| 10 | Enviar cotización     | Ventas           | Antonio Anaya       | 1        | 2            | 30/09/2026 (4 días restantes)     |
| 14 | Revisar metas del...  | Ventas           | Uziel Armando       | 1        | 3            | 30/09/2026 (4 días restantes)     |
| 21 | Aprobar gasto         | Finanzas         | Matias Hernandez    | 1        | 2            | 30/09/2026 (4 días restantes)     |
| 34 | Migrar servidor de... | Sistemas         | Calixto Isaac       | 4        | 5            | 26/09/2026 (vence hoy)            |
| 38 | Revisar contratos     | Recursos Humanos | Sin Asignar         | 2        | 2            | 26/09/2026 (vence hoy)            |
| 44 | Configurar firewal... | Sistemas         | Ana Torres          | 4        | 5            | 02/10/2026 (6 días restantes)     |
| 45 | Renovar contrato ...  | Finanzas         | Sin Asignar         | 3        | 2            | 31/12/2026 (96 días restantes)    |
| 46 | Renovar contrato ...  | Finanzas         | Sin Asignar         | 3        | 2            | 31/12/2026 (96 días restantes)    |
| 47 | Renovar póliza de ... | Finanzas         | Sin Asignar         | 3        | 2            | 15/10/2026 (19 días restantes)    |
| 48 | Renovar licencias ... | Finanzas         | Sin Asignar         | 3        | 2            | 26/09/2026 (vence hoy)            |

Departamento: Recursos Humanos 

Eliminar por ID

Buscar por Depto

Ordenar Tareas por Urgencia

## Caso de Prueba 21: Persistencia de la información

**Objetivo:** Verificar que las tareas activas y los empleados se conserven entre ejecuciones.

Pasos:

1. Anotar el número de tareas activas y de empleados.
2. Cerrar la aplicación desde el botón de cerrar de la ventana.
3. Volver a ejecutar la aplicación.
4. Registrar una tarea nueva.

**Resultado esperado:** Al reiniciar, la consola muestra "[BD] Datos cargados correctamente." y aparecen las mismas tareas (en sus mismas estructuras, con su responsable) y los mismos empleados. La nueva tarea recibe un ID mayor al último guardado. Las dependencias del grafo y los contadores de resueltas inician vacíos, tal como se describe en las decisiones de diseño.

SISTEMA AVANZADO DE GESTIÓN DE TAREAS ORGANIZACIONAL

Ver Todas (Ordenadas por Urgencia/Depto) Cambiar a modo oscuro

Menú Principal

Registrar Tarea

Pilas (Urgentes)

Colas (Programadas)

Listas (Generales)

Cola Prioridad

Empleados

Distribución de Tareas

Búsquedas

Grafo Dependencias

Ver Todas / Consola

| ID | Título                | Departamento     | Responsable Directo | Urgencia | Fecha de Entrega y Días Restantes | Estructura        |
|----|-----------------------|------------------|---------------------|----------|-----------------------------------|-------------------|
| 1  | Actualizar BD         | Sistemas         | Kevin Ramos         | 1        | 24/09/2026 (vence hace 2 días)    | Pila (Urgente)    |
| 2  | Reiniciar servidor    | Sistemas         | Ana Torres          | 3        | 30/09/2026 (4 días restantes)     | Pila (Urgente)    |
| 3  | Instalar antivirus    | Sistemas         | Calixto Isaac       | 2        | 30/09/2026 (4 días restantes)     | Pila (Urgente)    |
| 12 | Actualizar CRM        | Ventas           | Antonio Anaya       | 5        | 29/09/2026 (3 días restantes)     | Pila (Urgente)    |
| 18 | Generar nómina        | Recursos Humanos | Sin Asignar         | 5        | 29/09/2026 (3 días restantes)     | Pila (Urgente)    |
| 22 | Generar reporte ...   | Finanzas         | Galilea Valencia    | 4        | 30/09/2026 (4 días restantes)     | Pila (Urgente)    |
| 32 | Gestionar devoluc...  | Logística        | Joel Balderrama     | 5        | 29/09/2026 (3 días restantes)     | Pila (Urgente)    |
| 11 | Seguir pedido pen...  | Ventas           | Uziel Armando       | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 15 | Revisar currículums   | Recursos Humanos | Sin Asignar         | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 16 | Programar entrev...   | Recursos Humanos | Sin Asignar         | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 20 | Registrar factura     | Finanzas         | Matias Hernandez    | 1        | 28/09/2026 (2 días restantes)     | Cola (Secuencial) |
| 25 | Emitir recibo de p... | Finanzas         | Galilea Valencia    | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 28 | Actualizar inventa... | Logística        | Stephanie Lozano    | 2        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 29 | Recibir mercancía     | Logística        | Stephanie Lozano    | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 31 | Verificar stock       | Logística        | Joel Balderrama     | 1        | 30/09/2026 (4 días restantes)     | Cola (Secuencial) |
| 36 | Enviar reporte me...  | Ventas           | Carlos Ruiz         | 2        | 26/09/2026 (vence hoy)            | Cola (Secuencial) |
| 37 | Actualizar inventa... | Logística        | Stephanie Lozano    | 3        | 26/09/2026 (vence hoy)            | Cola (Secuencial) |
| 9  | Llamar a cliente p... | Ventas           | Carlos Ruiz         | 2        | 30/09/2026 (4 días restantes)     | Lista (General)   |
| 10 | Enviar cotización     | Ventas           | Antonio Anaya       | 1        | 30/09/2026 (4 días restantes)     | Lista (General)   |

Consola de Eventos en Tiempo Real

[BD] Datos cargados correctamente.

## Conclusión

El desarrollo del Sistema Empresarial Avanzado de Gestión de Tareas permitió integrar, en un caso de uso empresarial coherente, el conjunto de estructuras de datos y algoritmos fundamentales abordados a lo largo del curso: colas de prioridad implementadas mediante *heap* binario, árboles binarios de búsqueda para la organización de empleados y de tareas, tablas hash para el acceso en tiempo constante, algoritmos de ordenamiento y búsqueda (QuickSort, búsqueda binaria y búsqueda secuencial), técnicas de recursividad y divide y vencerás aplicadas al cálculo de estadísticas, a la distribución equilibrada de la carga de trabajo y al balanceo de árboles, y grafos dirigidos para el modelado de dependencias mediante ordenamiento topológico. La convergencia de estas estructuras bajo una arquitectura Modelo-Vista-Controlador evidencia que la selección de una estructura de datos no es una decisión aislada, sino una elección de diseño que debe responder a la naturaleza del problema que resuelve urgencia, orden de llegada, prioridad compuesta, relaciones jerárquicas o dependencias direccionales y articularse de manera coherente con las demás.

Un aporte particularmente relevante del proceso fue el ciclo iterativo de verificación, corrección y mejora aplicado sobre una base funcional ya construida. La corrección de defectos de implementación, como la condición que impedía la ejecución del caso base en la distribución por divide y vencerás o la falta de sincronización entre la cola de prioridad y su representación visual, ilustra que la correctitud de una estructura de datos no se valida únicamente por su definición teórica, sino por la verificación empírica de su comportamiento en cada punto de la aplicación que la consume. En las iteraciones posteriores, el sistema evolucionó de demostrar cada estructura por separado a hacerlas colaborar: el árbol de empleados alimenta el selector de responsables y la distribución por departamento; la tabla hash permite distinguir una tarea ya atendida de una inexistente; el árbol de tareas y el grafo se sincronizan automáticamente con las tareas activas; y la prevención de ciclos en el momento del registro sustituye a la simple detección posterior, reflejando una atención sostenida a la robustez del sistema.

Asimismo, la incorporación de la comparativa de métodos de búsqueda y del balanceo del árbol convirtió la complejidad algorítmica en un resultado observable: el propio sistema muestra que una búsqueda secuencial puede requerir tantas comparaciones como tareas existen, mientras que la tabla hash requiere una sola y un árbol balanceado del orden de  $\log_2 n$ . Esta evidencia, sumada a la persistencia inmediata en base de datos, a la validación estricta de datos de entrada y a una interfaz clara con visualizaciones del árbol y del grafo, demuestra que el rigor algorítmico y la usabilidad

no son objetivos contrapuestos, sino complementarios dentro de un mismo producto de software.

En términos generales, el proyecto satisface los objetivos planteados: implementa correctamente las estructuras y algoritmos avanzados requeridos, los documenta con un nivel de detalle que permite su comprensión y mantenimiento por terceros, optimiza las operaciones de asignación, consulta y organización de tareas mediante estructuras con una complejidad adecuada a cada operación, y permite una demostración funcional completa de cada componente. No obstante, resulta pertinente reconocer ciertas limitaciones que no comprometen la validez del sistema, pero delimitan su alcance actual: la comparación lexicográfica de los identificadores en el árbol de empleados, la ausencia de auto-balanceo en los árboles (el árbol de tareas se balancea bajo demanda), y que las dependencias del grafo y el historial de tareas atendidas no se conservan entre ejecuciones. Estas limitaciones constituyen líneas naturales de trabajo futuro, como la adopción de estructuras auto-balanceadas (AVL o rojo-negro) y la persistencia del grafo de dependencias.

En síntesis, el proyecto no solo cumple una función evaluativa dentro del curso de Estructuras de Datos, sino que constituye un ejercicio representativo de cómo la teoría algorítmica se traduce en decisiones de ingeniería concretas, sujetas a verificación, corrección y mejora continua, principios que resultan transferibles a cualquier contexto de desarrollo de software profesional.

## Referencias Bibliográficas

A continuación, se presentan las fuentes utilizadas para la explicación de los temas:

Erika Rossana Llanes Castro, Adelina Morita Alexander, & Lizzie Edmea Narváez

Díaz. (2026). Fortalecimiento del aprendizaje de estructuras de datos mediante

programación competitiva: un enfoque del constructivismo social. IE Revista de

Investigación Educativa de La REDIECH, 17.

[https://doi.org/10.33010/ie\\_rie\\_rediech.v17i0.2557](https://doi.org/10.33010/ie_rie_rediech.v17i0.2557)

Hernández, V., Guerra, L., y Gurrión, E. (2020). Estructuras de datos y algoritmos fundamentales. México: Tecnológico de Monterrey.

Gómez, A. (2023). Estructuras de datos. Clase 14-Colas de prioridad. Recuperado de <https://cs.uns.edu.ar/~mlq/ed/downloads/Teorias/uns-ed-2019-clase-14-Colas-con-prioridades.pdf>

[https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/1453/mod\\_resource/content/1/contenido/index.html#introduccion](https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/1453/mod_resource/content/1/contenido/index.html#introduccion)

<https://codegym.cc/es/groups/posts/es.291.estructuras-de-datos-pila-y-cola>

[https://pier.guillen.com.mx/algorithms/11-otros/11.4-estructuras\\_datos.htm](https://pier.guillen.com.mx/algorithms/11-otros/11.4-estructuras_datos.htm)